

MOTORES DE VIDEOJUEGOS: ESTUDIO Y APLICACIÓN

HANJIE ZHU

GRADO EN MATEMÁTICAS
FACULTAD DE CIENCIAS MATEMÁTICAS

UNIVERSIDAD COMPLUTENSE DE MADRID



TRABAJO FIN DE GRADO

DEPARTAMENTO DE SISTEMAS INFORMÁTICOS Y COMPUTACIÓN

Junio 2022

Director: Fernando Rubio Diez

Autor: Hanjie Zhu

ÍNDICE GENERAL

RESUMEN & MOTIVACIÓN	3
INTRODUCCIÓN	4
CAPÍTULO 1: CONOCIENDO CON UNITY.....	5
1.1 – INTRODUCCIÓN A UNITY.....	5
1.2 – CARACTERÍSTICAS DE UNITY.....	5
1.3 – INTERFAZ DE UNITY.....	7
1.4 – EDITORES DE SCRIPTS.....	8
1.5 – MOTOR FÍSICO.....	9
CAPÍTULO 2: CREANDO UN PLAYER PASO A PASO.....	12
2.1 – IMPORTACIÓN DE ASSERT EN UNITY.....	12
2.2 – DISEÑO DE LA ESCENA Y LAYER.....	13
2.3 – CREACIÓN DE PLAYER Y SU MOVILIDAD.....	15
2.4 – ANIMACIÓN DE PLAYER Y LAYERMASK.....	18
EXTRA 1 – DEBUG: ERRORES MÓVILES Y MEJORAR SCRIPT.....	22
2.5 – CONTROL DE CÁMARA Y COLECCIÓN DE ARTÍCULOS.....	23
EXTRA 2 – INTRODUCCIÓN AL USER INTERFACE	26
CAPÍTULO 3: 3E:ENEMIGOS, ENGENDRADOR Y EFECTOS.....	28
3.1 – CREACIÓN DE ENEMIGOS.....	28
3.2 – SPAWNER(ENGENDRADOR).....	31
3.3 – EFECTOS: AUDIO, DIALOGO Y LUZ.....	32
CAPÍTULO 4: SCENEMANAGER Y MENUS.....	39
4.1 – SCENEMANAGER.....	39
EXTRA 3 – DEBUG: INFLEXIBILIDAD DE JUMP.....	40
4.2 –MAINMENU Y PAUSEMENU.....	41
CAPÍTULO 5: ONLINE MULTIPLAYER: PHOTON PUN2.....	45
5.1 – INTRODUCCIÓN A PHOTON.....	45
5.2 – CREACIÓN DEL MODELO MULTIJUGADOR.....	45
5.3 –LOGIN&NAME.....	49
CAPÍTULO 6: CONCLUSIONES.....	51
6.1–LAS DESVENTAJAS Y VENTAJAS DE UNITY.....	51
6.2 –PRINCIPANTES USANDO UNITY.....	54
BIBLIOGRAFÍA	56

RESUMEN

El objetivo de este trabajo es aprender los procedimientos básicos necesarios para el desarrollo de videojuegos utilizando distintas plataformas que existen actualmente en internet, principalmente como Unity, Godot o Unreal Engine. Inicialmente, se definen los diferentes aspectos y componentes de las plataformas de desarrollo de juegos, de forma que se van explicando sus usos básicos (en qué lugar) y la aplicación de ellos.

En la segunda parte, se puede ver más interesante ya que se va desarrollando un juego para poder entender de forma más profunda los aspectos que se han explicado anteriormente. Durante el proceso serán necesarios muchos conocimientos sobre el lenguaje de programación C#, por lo tanto, se irán explicando las diferentes funciones aplicadas en el desarrollo de juegos usando C#, especialmente en la librería de UnityEngine.

Finalmente, se desarrollará un videojuego, usando Build Settings, en una plataforma.

ABSTRACT

The objective of this work is to learn the basic procedures necessary for the development of video games using different platforms that currently exist on the internet, mainly such as Unity, Godot or Unreal Engine. Initially, the different aspects and components of game development platforms are defined, so that their basic uses (where) and their application are explained.

In the second part, it can be seen as more interesting since a game is being developed to be able to understand in a deeper way the aspects that have been explained previously. During the process, a lot of knowledge about the C# programming language will be necessary, therefore, the different functions applied in the development of games using C# will be explained, especially in the UnityEngine library.

Finally, a video game will be developed, using Build Settings, on a platform.

MOTIVACIÓN

Actualmente, las herramientas como Unity o Unreal Engine no son plataformas solo para el desarrollo de videojuegos. También pueden usarse en varias áreas como arquitectura, astronomía, diseños de cualquier tipo, etc. Debido al desarrollo de la tecnología en estos 30 años, estas herramientas son poderosas hoy en día. Teniendo en cuenta esto, decido aprender a usarlas para el futuro desarrollo del mundo metaverso. Con una alta posibilidad, los futuros juegos, trabajos o la mayoría de las actividades se realizarán en el metaverso (una realidad virtual), donde es posible simular el mundo real. Por este motivo, realizo este trabajo para poder poner en contacto los motores de videojuegos y también el lenguaje de programación con funciones aplicables en la librería, C#.

INTRODUCCIÓN

El motor de videojuegos se refiere a la agrupación de los componentes utilizados en el desarrollo de videojuegos (para el diseño, la creación y la comprobación de un juego), que están contruidos por una serie de rutinas de programación. Además, la mayoría de ellos son aplicables para algunas aplicaciones de imágenes interactivas en tiempo real.

Estos sistemas proporcionan a los diseñadores de juegos varias herramientas necesarias para escribir juegos, y su propósito es permitir que los diseñadores de juegos creen programas de juegos de una forma fácil y rápida, sin tener que comenzar desde cero. La mayoría de ellos son compatibles con múltiples plataformas de sistemas operativos, como Linux, Mac OS X y Windows.

La mayoría de los motores de juegos incluyen los siguientes sistemas: el motor de renderizado (es decir, "renderizador", incluido el motor de imágenes 2D y el motor de imágenes 3D), el motor de física, el sistema de detección de colisiones, los efectos de sonido, el motor de secuencias de comandos, la animación por computadora, la inteligencia artificial, el motor de red y la gestión de escena, etc.

El sistema de renderizado del motor 3D generalmente se basa en un conjunto de API de imágenes de alto nivel, como Direct3D u OpenGL, que encapsulan algunas de las funciones de la GPU y la tarjeta gráfica. También se utilizan unos de niveles inferiores, con una menor frecuencia, como DirectX, SDL y OpenAL, que proporcionan soporte independiente para otros dispositivos de hardware, como dispositivos de entrada (mouse, teclado, joystick), tarjetas de red y tarjetas de sonido. Antes de la llegada de las tarjetas gráficas aceleradas por hardware, los desarrolladores usaban el renderizado por software. Ahora el renderizado por software se usa ampliamente para renderizar imágenes que no son oportunas, o para el dispositivo de hardware del usuario que no es compatible con el renderizado por hardware.

El desarrollo de motores de juegos es una profesión relativamente popular, enfatiza la cooperación mutua entre disciplinas, como geometría, teoría del color, programación de computadoras, etc. Principalmente en el campo de las ciencias visuales. Algunas empresas incluso utilizan motores de desarrollo como actividades divertidas y técnicas en lugar de venderlos por dinero. Por ejemplo, Godot y Blender son conocidos motores de juegos multiplataforma de código abierto gratuitos.

Con respecto a su historia, no entramos en ello ya que es algo no necesario para aprender cómo funciona un tipo de motor de videojuego para la creación de un juego. Para los que estén interesados en su historia, recomiendo leerla en Wikipedia que está claramente especificada. [Referencia, https://es.wikipedia.org/wiki/Motor_de_videojuego#Historia].

A continuación, entramos en la parte del desarrollo de este trabajo, que en principio va a tratar de jugar con Unity. Y una vez que conozcamos Unity, para profundizar más, se pondrá el énfasis en el estudio de juegos multijugadores usando Photon Unity Networking 2 (PUN2). Photon Unity Networking es un paquete de complemento multijugador de Unity. Brinda opciones de autenticación, emparejamiento y comunicación rápida y confiable en el juego a través del backend Photon.

Capítulo 1

Conociendo Unity

1.1-Introducción a Unity

Definición. Unity es un tipo de motor de videojuegos con el que es posible desarrollar juegos de 2D/3D, creado por Unity Technologies para diversas plataformas como Windows, MacOS X, Android, IOS, XBOX ONE, PS4, Nintendo Switch, Wii, Nintendo 3DS, WebGL. También puede utilizar el complemento Unity Web Player para publicar juegos web.

Más aún, se puede aplicar en varios campos usando su habilidad de creación de 3D, AR(augmented reality) y VR(virtual reality). Por ejemplo, aplicando en la educación en simulación virtual, la industria militar y aeroespacial, el diseño interior, la planificación de la ciudad, la exposición de reliquias culturales, etc.

1.2-Características de Unity

1. Multiplataforma

Los desarrolladores de juegos pueden desarrollar a través de diferentes plataformas. Una vez que se completa el juego, el juego se puede publicar directamente en las plataformas principales de uso común con un clic sin ninguna modificación.

Como vemos, el desarrollo multiplataforma puede ahorrar mucho tiempo a los desarrolladores de juegos. En el pasado, en el desarrollo de juegos, los desarrolladores tenían que considerar las diferencias entre plataformas, como el tamaño de la pantalla, el modo de operación, las condiciones del hardware, etc, que afectarían directamente el progreso del desarrollo y causarían grandes problemas a los desarrolladores. Unity es casi perfecto para los desarrolladores. Resolver este problema reducirá en gran medida los problemas innecesarios en el proceso de trasplante.

2. Edición general

La interfaz de usuario de Unity tiene edición visual, editor de atributos detallado y funciones de vista previa dinámica del juego. El innovador modo de visualización de Unity permite a los desarrolladores de juegos crear fácilmente experiencias interactivas. Cuando el juego se

está ejecutando, los valores de los parámetros se pueden modificar en tiempo real, lo que facilita el desarrollo y ahorra mucho tiempo para el desarrollo del juego.

3. Importación de recursos

Los proyectos pueden importar recursos automáticamente y actualizarse automáticamente de acuerdo con los cambios de recursos. Unity admite casi todos los formatos 3D convencionales, como 3ds Max, Maya, Blender, etc. El material de textura se convierte automáticamente al formato U3D y puede funcionar en coordinación con la mayoría de las aplicaciones relacionadas.

4. Implementación con un clic

Unity puede completar el desarrollo y la implementación multiplataforma de trabajos con un solo clic, lo que permite que los trabajos de los desarrolladores se presenten en múltiples plataformas.

5. Los lenguajes de programación

Unity integra la plataforma de compilación MonoDeveloper y es compatible con los lenguajes de scripting C#, JavaScript y Boo. Entre ellos, C # y JavaScript son los lenguajes de scripting más utilizados en el desarrollo de juegos. En nuestro caso, desarrollamos scripts en el lenguaje C# y explicando el uso de las funciones de la clase MonoBehaviour.

6. Online

Unity permite el desarrollo de juegos independientes y también a los juegos online de multi-jugadores masivos.

7. Shader

El Shader de Unity integra facilidad de uso, flexibilidad y alto rendimiento.

8. Editor de terreno

Unity 3D tiene un potente sistema de edición de terreno incorporado, que permite a los desarrolladores del juego implementar cualquier terreno complejo en el juego, admite la creación de terreno y el parcheo de árboles y vegetación, admite LOD de terreno automático, efectos especiales de superficie de agua, especialmente hardware de gama baja. El buen funcionamiento del vasto y exuberante paisaje de vegetación puede crear fácilmente varios terrenos utilizados en las escenas del juego.

9. Efectos físicos

El motor de física es un programa de computadora que simula el modelo mecánico newtoniano, que utiliza variables como masa, velocidad, fricción y resistencia del aire. Unity incorpora el motor de física PhysX de NVIDIA, los desarrolladores de juegos pueden restaurar y simular efectos físicos del mundo real de una manera eficiente, realista y vívida, como detección de colisiones, efectos de resorte, efectos de tela, efectos de gravedad, etc.

10.Luz y Sombra

Unity proporciona un sistema de renderizado de luces y sombras con sombras suaves y efectos de horneado altamente sofisticados.

1.3-Interfaz de Unity

Unity tiene una poderosa interfaz de edición, los desarrolladores de juegos pueden crear juegos a través de la interfaz de edición visual. Por defecto, tiene una forma como la siguiente imagen.

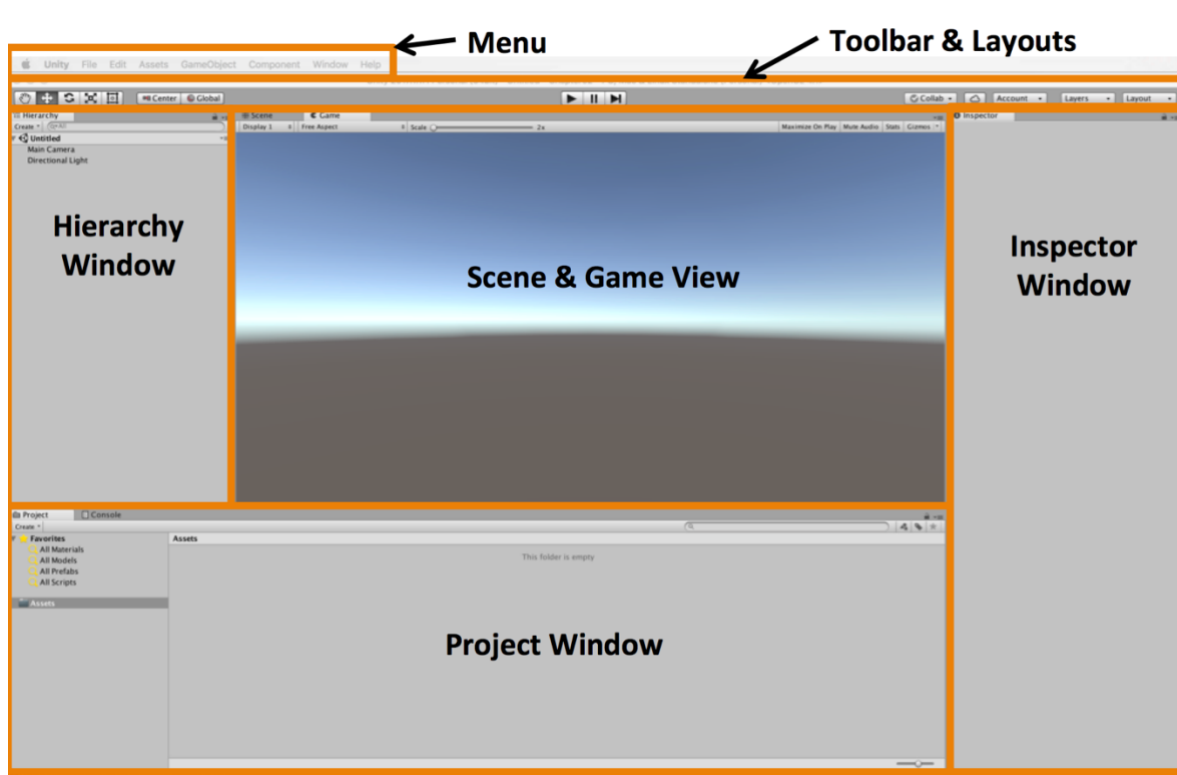


Figure 1.1. Interfaz de Unity

1. Scene

Es un monitor visual donde se puede ver el aspecto del juego que se está desarrollando. Puede moverse usando “Hand Tool” de la barra de herramientas de la siguiente imagen.

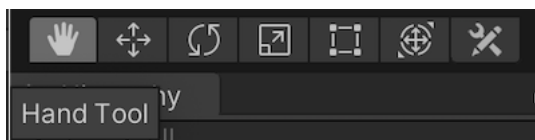


Figure 1.2. La barra de herramientas de movilidad.

2. Hierarchy

En principio está situada en la parte izquierda de la interfaz de Unity, donde se pueden encontrar los elementos que existen dentro del juego desarrollado. Cuando se crea el proyecto va a generar por defecto dos elementos: Main Camera (cámara que permite visualizar el juego, es como la cámara de una película) y Directional Light (objeto que permite iluminar los elementos del juego, parecido al light de una película).

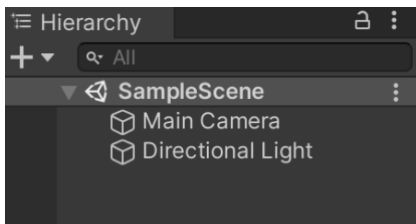


Figure 1.3. Hierarchy.

3. Project

Lugar donde se importan y guardan los elementos del juego. Normalmente a estos elementos los llaman “Assets”.

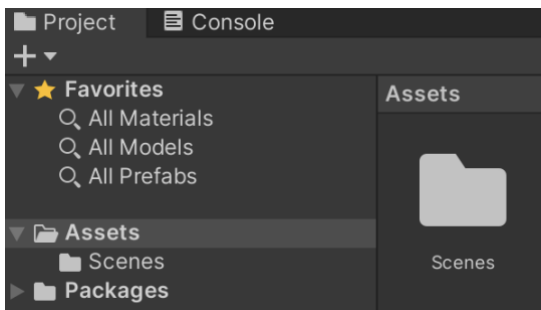


Figure 1.4. Project

4. Inspector

El Inspector se utiliza para mostrar información detallada sobre todos los complementos (los scripts pertenecen a componentes) del objeto de juego seleccionado actualmente y sus propiedades.

1.4-Editores de Scripts

.....

1. La ventana consola

La ventana consola permite visualizar los resultados después de ejecutar a un script de códigos. También alerta sobre los errores en el código de programación. Normalmente, utilizo Visual Studio para editar el código de C#.

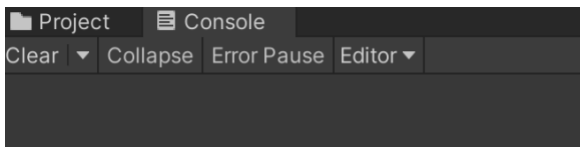


Figure 1.5. La ventana consola

(Hay que seleccionar Preferences → External Tools → External Script Editor: Visual Studio para poder asignar VS como editor del código para el juego. También pueden usarse otros editores para el código de programación)

1.5-Motor Físico

.....

El Motor físico de Unity utiliza el procesador de física acelerado por hardware PhysX, específicamente responsable de los cálculos físicos. Por lo tanto, el motor de física de Unity es más rápido y también puede reducir la carga de la CPU.

El motor físico de Unity incluye principalmente cuerpos rígidos, colisiones, materiales físicos y movimientos de articulaciones.

La función del motor físico en el juego es simular la interacción entre objetos cuando una fuerza externa actúa sobre el objeto. Es decir, el motor físico se utiliza aquí para simular el efecto real posterior a la colisión. A través del motor físico, el efecto de la influencia mutua entre los objetos es bastante simple.

1. Rigidbody

El Rigidbody (cuerpo rígido) en Unity puede asignar propiedades físicas a los objetos del juego, de modo que los objetos del juego puedan recibir empuje y torsión bajo el control del sistema físico, logrando así efectos de movimiento en el mundo real. Las propiedades del cuerpo rígido incluyen la masa, el arrastre, el arrastre angular, la gravedad, ser cinemático y la detección de colisiones.

Inspector → Component → Physics (Physics 2D) → Rigidbody (Rigidbody 2D).

ATENCIÓN: Debe agregarse el componente rígido del cuerpo al objeto para que pueda ser afectado por sus propiedades.

2. Collider

Durante el proceso del desarrollo de un juego, los objetos del juego deben interactuar con los atributos físicos de acuerdo con sus necesidades. Para ello, los componentes físicos de Unity proporcionan el componente Collider. Collider es un tipo de componente físico, que junto con el cuerpo rígido hace que se produzca la colisión.

Un cuerpo de colisión normalmente tiene aspecto como un cuadrado, una esfera o una cápsula, que debe tener sentido con la forma de su GameObjects (un objeto en Unity).

Inspector → Component → Physics (Physics 2D) → Box/Sphere/Capsule/Mesh/Wheel/Terrain Collider.

3. Trigger

En Unity, hay dos formas de detectar colisiones, una es usar Collider y la otra es usar Trigger. El Trigger se usa para desencadenar los eventos. Por ejemplo, en un juego de rol, cuando el jugador camina hacia un lugar donde ocurrirá un evento de Boss, se puede usar Trigger para lograrlo.

Cuando el objeto del juego vinculado con el colisionador entra en el área de activación, se ejecutará la función OnTriggerEnter (así activa un evento).

ATENCIÓN: La casilla de verificación IsTrigger debe estar marcada en el componente colisionador en el inspector.

La detección de información de Trigger utiliza las siguientes 3 funciones:

- MonoBehaviour.OnTriggerEnter(Collider collider) si está activando los eventos.
- MonoBehaviour.OnTriggerExit(Collider collider) si está desactivando los eventos.
- MonoBehaviour.OnTriggerStay(Collider collider) si mantiene activado los eventos.

La diferencia entre Collider y Trigger es que el primero es el portador del segundo, y el segundo es solo un atributo del primero.

Si se desea detectar el contacto de un objeto, pero no se desea que la detección de colisión afecte al movimiento del objeto, o si se desea detectar si un objeto pasa a través de un área determinada en el espacio, entonces puede usarse un disparador. Por ejemplo, el collider es para simular el efecto del golpeo de un automóvil, una pelota que cae y rebota en el suelo, y el Trigger es como para simular el efecto de la puerta que se abre automáticamente cuando una persona se para cerca de la puerta.

4. AddForce

La función AddForce afecta a la forma en que se mueve su GameObject al permitirle definir su propio vector de fuerza, así como elegir cómo aplicar esta fuerza al GameObject (este GameObject debe tener un componente Rigidbody adjunto).

Usando ForceMode permite elegir entre cuatro formas diferentes de afectar el GameObject: Aceleración, Fuerza, Impulso y VelocityChange.

<u>Force</u>	Add a continuous force to the rigidbody, using its mass.
<u>Acceleration</u>	Add a continuous acceleration to the rigidbody, ignoring its mass.
<u>Impulse</u>	Add an instant force impulse to the rigidbody, using its mass.
<u>VelocityChange</u>	Add an instant velocity change to the rigidbody, ignoring its mass.

5. Character Controller

La componente que permite controlar el movimiento de un GameObject. Además, permite moverse mientras afecta a una colisión sin usar el rigidbody.

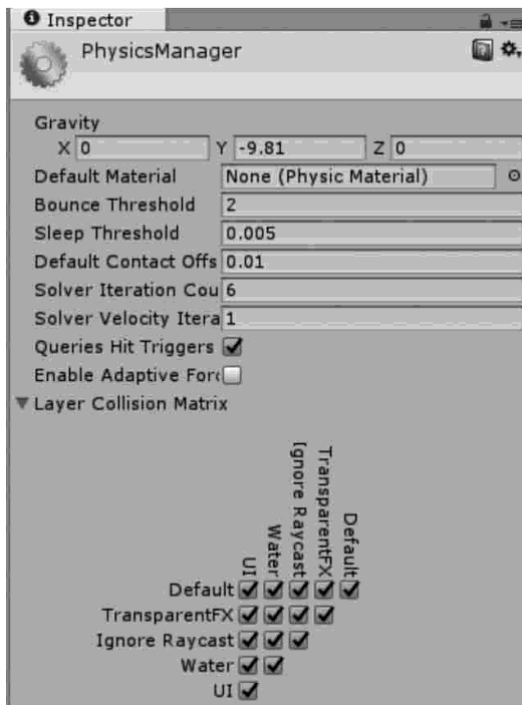
Inspector -> Component -> Physics (Physics 2D) -> Character Controller.

6. Ajuste especial: Physics Manager

Como excelente plataforma de desarrollo de juegos, el entorno de desarrollo integrado Unity proporciona un modo de gestión excelente, llamado Physics Manager.

Physics Manager administra los parámetros de los efectos físicos en el proyecto, como la gravedad del objeto, la fuerza de rebote, la velocidad y la velocidad angular.

Pueden cambiarse los efectos físicos en el juego ajustando los parámetros en el administrador de física según sea necesario.



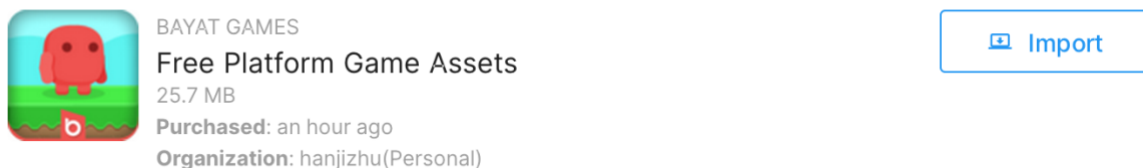
Capítulo 2

Creando un Player paso a paso

Una forma de poder entender de una forma adecuada el uso de las funciones y los componentes mencionados anteriormente es ir desarrollando un juego paso a paso por nuestra propia cuenta. Por lo tanto, en este capítulo se va a ir desarrollando un videojuego (y mientras tanto, explicando) en Unity de tipo 2D que es análogamente para el de tipo 3D y cualquiera de otros tipos.

2.1- Importación de Asset en Unity

El primer paso para poder empezar a crear un videojuego es importar los posibles recursos que va a ir usando durante todo el proceso, denotando estos recursos como “asset”. Para ellos podemos usar la importación a mano: Menu → Assets → Import New Asset y seleccionar el archivo que contiene los recursos. O bien podemos buscar assets en Unity Asset Store e importar en el store. Entonces el primer paso va a ser importar el siguiente paquete de assets en el store, que una vez descargado, pulsa import y ya lo tiene.



Version: 1.9.4 • Mar 9, 2020

Fixed 2D water animation Frames. Thanks to Dave for suggest it.

Figure 2.1. El paquete de asset descargado y guardado en mi cuenta de Unity.

Importante. Hay que tener la versión de 2019 de Unity abierta para la importación del paquete de asset. Teniendo en cuenta que la versión de 2020 o una superior no deja importar este paquete.

Aparentemente, se aparecerá la mostración de todos los assets en este paquete y podrá seleccionar los que quiere importar para el proyecto. Y después de haber seleccionado, se hace click en import y luego esperaremos (dependiendo del tamaño) hasta que aparezca en la ventana de Project.

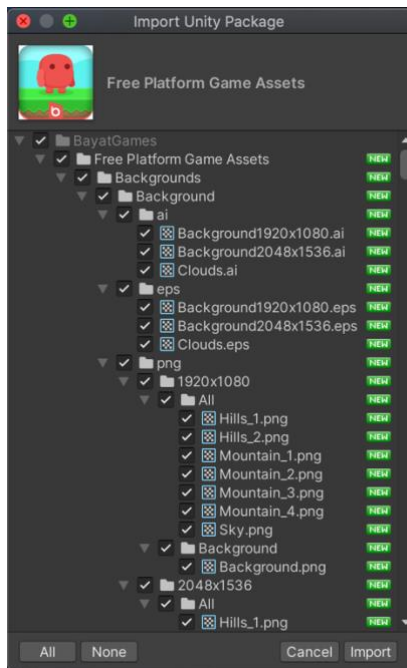


Figure 2.2. Presentación de todos los assets en el paquete gratuito del autor “BAYAT GAMES” en Unity Asset Store.

2.2-Diseño de la escena y Layer

En general, la mayoría de los desarrolladores de juegos empieza con la disposición de la escena. Que sigue los siguientes pasos: ajustar el tamaño del monitor, insertar el background en la escena, diseñar y construir el terreno y el medio ambiente del juego.

Paso 1. En la ventana de scene selecciona Game y luego intercambia la resolución del juego a 1920x1080 (también se pueden usar otras, pero para este juego se prefiere con este tamaño). Así, se ajusta al tamaño del monitor apropiado.



Figure 2.2. Parte de la ventana scene donde es para seleccionar y añadir los diferentes Tab.

Paso 2. Arrastrar el fondo que se encuentra en el fold asset → BayatGames → Free Platform → Background (hay tres tipos) a Hierarchy, y lo mismo para el terreno (en el fichero de Tiles) y el medio ambiente (en fichero Enviroment). Después se va diseñando la escena con la barra de tolos que tiene en la esquina superior izquierda (Figura 1.2).

Importante. En este segundo paso llega el problema de que el fondo está cubriendo los otros gameObject (los objetos que se encuentran en la escena, en el Hierarchy), ya que fue implementado antes que la escena. Para resolver esto, utiliza Inspector → Sprite Renderer → Sorting Layer (hay otro Layer que se encuentra en la parte superior de Inspector, que es para el uso de distinguir los diferentes objetos, no es lo mismo que el que se usa en este apartado).

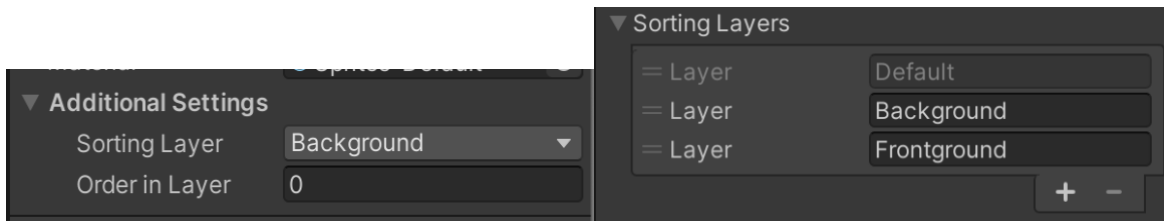


Figure 2.3. Sorting Layer

Paso 3. Para resolver el problema anterior, se añaden dos layers (Figure 2.3) que tienen preferencia sobre el último añadido (o bien usar Order in Layer, que tiene preferencia sobre los gameObject con número mayor), luego cambiando el Sorting Layer de cada gameObject que tienen que aparecer por delante del fondo. De esta forma, ya puede ir diseñando la escena del juego.

Al final de estos pasos, llega a la escena de la forma de mi gusto como Figure2.4.

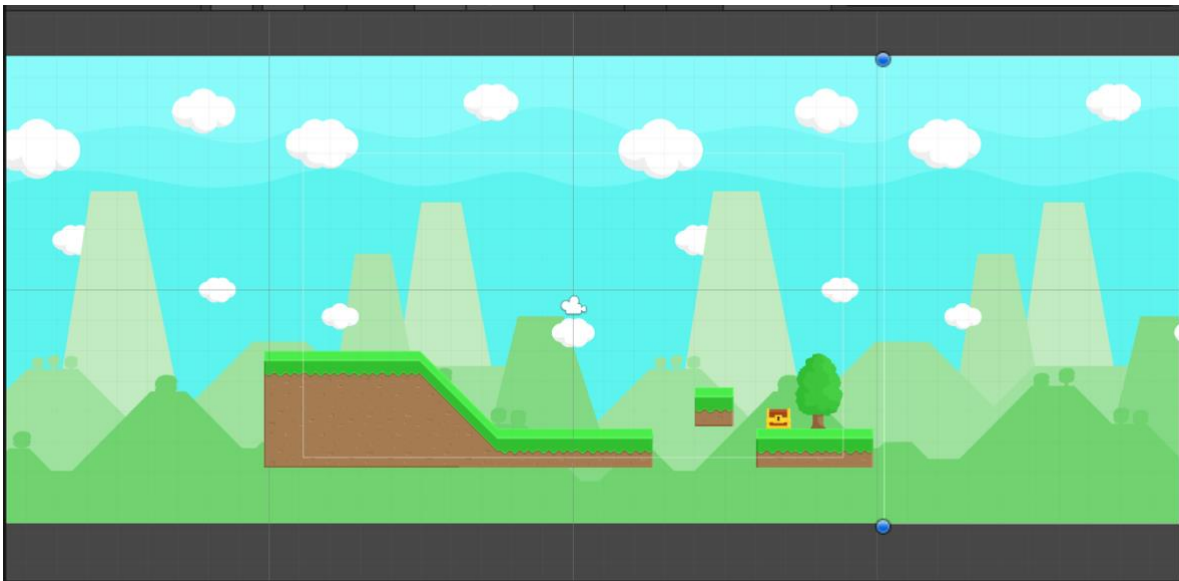


Figure 2.4. La escena básica.

Prefabs. Después del diseño de la escena, para modificar los elementos que son réplicas de un asset, no es necesario hacerlo de uno en uno gracias a prefabs. Prefabs es un fichero (creado por el desarrollador) en el que al añadir (antes del diseño de la escena) un asset podrá concatenar los estados de sus duplicados (poder modificar los estados de réplicas en el Inspector de la prefab, se recomienda hacer prefabs de cada asset si es necesario usar sus réplicas).

Otra recomendación. Es mejor crear un gameObject vacío (Hierarchy → clic ratón derecho → Create Empty) para guardar los assets de mismos tipos y sus réplicas (en mi caso, lo diferencia en el fondo, el terreno y el medio ambiente).

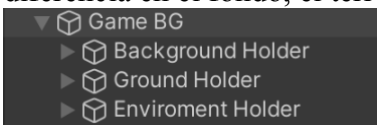


Figure 2.5. El soporte de cada tipo de assets.

2.3-Creación de Player y su movilidad

En esta sección se van a seguir 4 pasos para la creación de un player, que son los siguientes:

Paso1. Fijando en el fichero Character → Character Animation → Idle, se encuentra una imagen de un carácter con 11 formas diferentes (Figure 2.6), pero en esta fase solo necesita la primera.

Para obtenerla se hace clic en la imagen → selecciona Multiple en Sprite Mode → clic en Sprite Editor, sale un panel con la imagen y se puede aplicar de dos formas: a mano o automáticamente.

- La primera forma es usando el ratón para cuadrarlos de uno en uno.
- La otra forma es usar la poderosa editora de imagen de Unity de forma automática, seleccionando Slice → Type: automatic → slice (Figure 2.6)

Luego, se hace clic en Apply y nos llega a 11 diferentes imágenes del carácter del juego.



Figure 2.6. Separación en formato imagen.

Paso 2. Crear un gameObject en Hierarchy con el componente Sprite Renderer en Inspector (de forma clic ratón derecho → 2D Object → Sprite, o bien crear un Empty Object y añadir componente Sprite Renderer), que permite renderizar la imagen en la escena al arrastrar la imagen en Sprite. De esta forma, el carácter aparece en la ventana Scene. Pero si prueba con el botón de Play, el carácter no hace nada (queda en el lugar donde se pone).

Así que tiene que añadir componentes y scripts para que funcione bien. Lo primero es añadir en Inspector el componente Rigidbody 2D (en el caso de un juego 3D, se usa Rigidbody) para que tome la gravedad y el componente Box Collider 2D para poder colisionar con otros gameObject que tienen este componente (Figure 2.5).

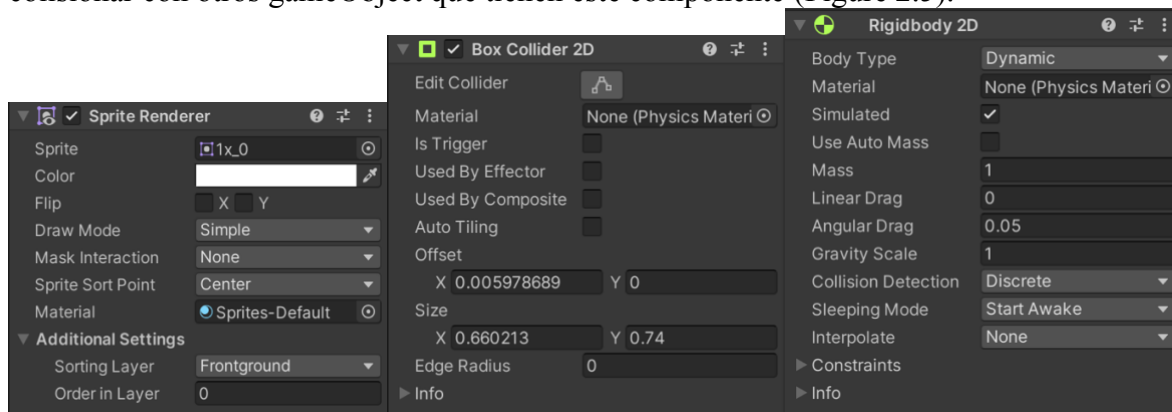


Figure 2.5. Los componentes que ofrece Unity se encuentran en Inspector → Add Component.

Observación. Es claro que necesita también agregar el componente Tipo Collider (dependiendo de la forma de ese gameObject) para cada gameObject que pertenece al terreno, si no es posible que el carácter colisione con el suelo.

- **En lo siguiente necesita crear el script para movilizar el carácter.**

Entra a la segunda parte de esta sección de las movilidades de un carácter: movimiento de izquierda, derecha, y salto.

Extra: Script1. Introduciendo unos conocimientos de uso de la librería UnityEngine para poder entender bien lo que se va a tratar a continuación.

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class NewBehaviourScript : MonoBehaviour
6  {
7      // Start is called before the first frame update
8      void Start()
9      {
10
11      }
12
13      // Update is called once per frame
14      void Update()
15      {
16
17      }
18  }

```

Figure 2.6. Un nuevo C# script creado dentro de Unity.

Para poder usar el script, se necesita crear en la ventana Project un fold de C# script y adjuntar al gameObject que vaya a tener las propiedades que define en este script.

En principio, se ve en la figura 2.6 que tiene una clase pública NewBehaviourScript heredada con la clase MonoBehaviour suministrada por la librería UnityEngine. En la clase MonoBehaviour ya tiene todas las propiedades y las funciones básicas definidas que podemos ver a lo largo de este capítulo. Por este momento, solo vamos a tratar de entender las siguientes:

1. Función void Update()

Actualización normal, esta función se agrega de forma predeterminada al crear un script C#, y el sistema ejecuta la función **una vez en cada fotograma**. Y la actualización está relacionada con la cantidad de fotogramas de la plataforma actual.

2. void FixedUpdate()

FixedUpdate se ejecuta en un intervalo de tiempo fijo y no se ve afectado por la velocidad de fotogramas de la plataforma. Por lo tanto, es mejor usar FixedUpdate cuando se trata de Rigidbody.

3. void Awake()

Activación del script, que se utiliza para la inicialización del script, que se ejecuta una vez en el ciclo de vida del script.

4. void Start()

Ejecutar antes de Update () y después de Awake (). La diferencia entre la función Start () y la función Awake () es que la función Start () solo se ejecuta cuando el script está habilitado y la función Awake () se ejecuta a la inicialización de script independientemente de si el script está habilitado o no.

Extra. Para activar y desactivar el script solo hace falta pinchar ☒ en la parte del Inspector de Script.

5..GetAxis, GetAxisRaw y GetButtonDown

GetAxis es una función que nos devuelve el valor entre 0 y 1, o 0 y -1, dependiendo de la tecla que pulsa. Por ejemplo, GetAxis("Horizontal") devuelve valor en [0,1] si pulsa la tecla A (por defecto) y devuelve valor en [0,1] si pulsa la tecla D (por defecto).

La diferencia de GetAxis y GetAxisRaw es que GetAxisRaw solo devuelve el valor 1,0,-1 exactamente.

La función GetButtonDown devuelve True si tiene pulsado la tecla indicada y en el caso contrario devuelve False.

Paso3. En este paso, utiliza las funciones vistas anteriormente para hacer mover y saltar el carácter creado antes.

Lo primero es crear una función Movement() para agregar las propiedades en él y tiene que estar en la función FixedUpdate() para que funcionen bien las operaciones que hacen en la variable body.

Después hay que crear las variables body, moveForced, jumpForce y hace visualizar las dos últimas en Inspector usando [SerializeField]. Es necesario agregar esto para poder visualizar en el Inspector las dos variables privadas.

ATENCIÓN: La variable body tiene que ser pública para poder agregar el Rigidbody2D del carácter en el Inspector.

La función movement es fácil de entender una vez que ya sabemos las funciones 1 hasta 5. Se puede decir en palabras como sigue: Considerar acceso a los teclados de movimientos, si tiene pulsado algunos de ellos entrará al if-statement que viene después, y hacer movimiento para cambiar su velocidad (en el componente Rigidbody) en el caso de horizontal usando Vector2 (vector de dimensión 2), cambiar la escala local (que está en el componente Transform) del carácter para poder orientar a la dirección correcta y hacerse saltar al cambiar la velocidad vertical usando Vector2 (vector de dimensión 2).

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class PlayerController : MonoBehaviour
6  {
7      public Rigidbody2D body;
8      [SerializeField]
9      private float moveForced;
10     [SerializeField]
11     private float jumpForce;
12
13     // Start is called before the first frame update
14     void Start()
15     {
16     }
17
18     // Update is called once per frame
19     void FixedUpdate()
20     {
21         Movement();
22     }
23
24     void Movement()
25     {
26         float horizontalmove = Input.GetAxis("Horizontal");
27         float facedirection = Input.GetAxisRaw("Horizontal");
28
29         if (horizontalmove != 0)
30         {
31             body.velocity = new Vector2(horizontalmove * moveForced * Time.deltaTime, body.velocity.y);
32         }
33         if (facedirection != 0)
34         {
35             transform.localScale = new Vector3(facedirection, 1, 1);
36         }
37
38         if (Input.GetButtonDown("Jump"))
39         {
40             body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
41         }
42     }
43 }

```

Figure 2.7. Script de movimientos del carácter.

2.4-Animación de Player y LayerMask

Los que juegan a los juegos, seguramente saben que el carácter siempre viene con la animación cuando está en reposo, en camino, o saltando. En esta sección vamos a ver cómo puede hacer las tres animaciones en el carácter usando Unity. También es posible hacer assets que provienen de otra plataforma e importar directamente en Unity.

Paso1. Ajuste de los assets.

Visto en el primer paso de la sección 2.3, que necesita hacer slice para cada imagen de Blink, Jump y Run. Que se encuentran en el fichero de Character y que va a usar después.

Paso2. En lo siguiente van a hacer procesos en el componente Animator y Animation. Agregando el componente Animator en el gameObject Player, creando el fichero para guardar nuevos tipos de assets (Animator Controller y Animation).

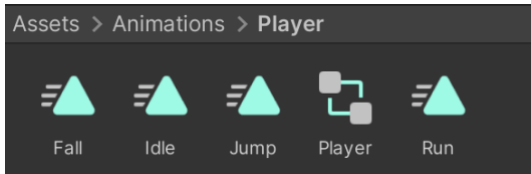


Figure 2.8. Cuatro Animation para la animación de reposo, salto, caída y ejecución, y un Animator Controller.

Paso3. Entra en la ventana Animation Idle, arrastra cada fotograma en la ventana y ajusta bien el ritmo de la animación usando  de la ventana Animation, así para cada animación. Luego, llega a lo difícil de configurar bien los elementos de Animator.

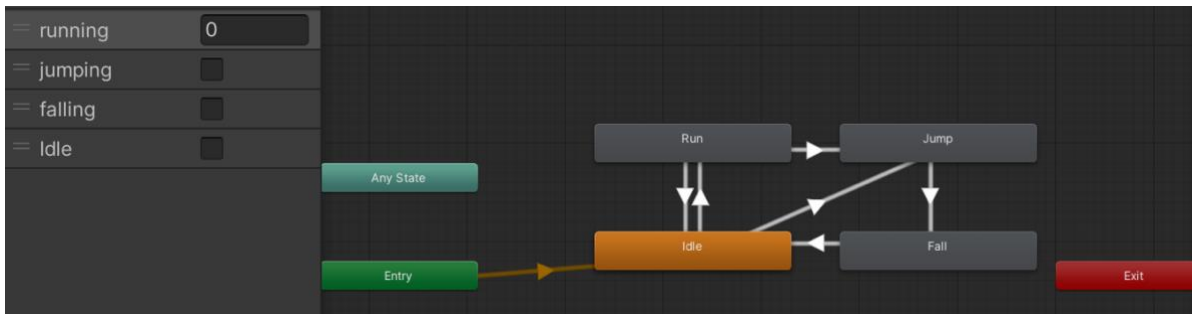


Figure 2.9. Animator de Player.

En esta figura 2.9 se trata de implementar las relaciones entre cada animación. Se puede entender como: Cuando entra en el juego (Entry), el Player queda en reposo (Idle) y puede hacer cambios entre Idle y Run (entra a la condición de running) sin tener ninguna condición (solo con el teclado), poder saltar cuando el carácter está en reposo o en ejecución (entrando a la condición de jumping), cambia a la acción Fall cuando llega al punto máximo de salto (entrando a la condición de falling) y al colisionar con el suelo transforma directamente en Idle (entra a la condición de falling).

Para hallar esto necesitamos crear transformaciones entre ellos (que son las flechas blancas de la imagen 2.9) para lo cual se hace click derecho en el ratón, se selecciona Make Transition y se arrastra en el otro elemento. Y para cada uno de ellos hacer que sus setting se queda como (en Inspector):

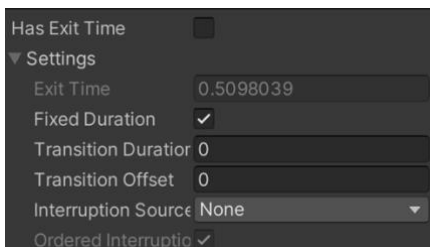
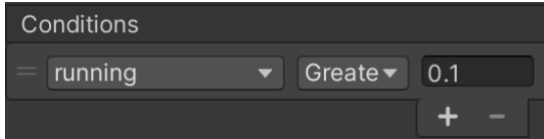


Figure 2.11. Setting de cada Transition.

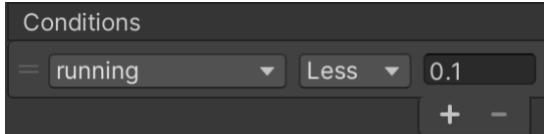
Importante. Tiene que añadir condiciones en cada transición en el Inspector.

Condición de transición Idle → Run.



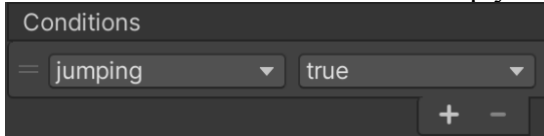
Tienen lógica entendida como si el valor es mayor que 0.1 (de.GetAxis), entonces está en ejecución. En caso contrario, en reposo.

Condición de transición Run → Idle.



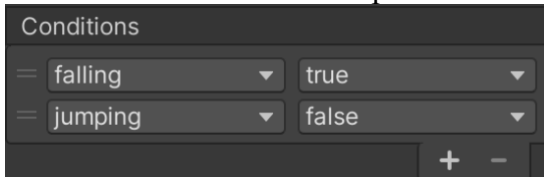
En caso contrario, en reposo (running es variable float)

Condición de transición Idle → Jump y Run → Jump.

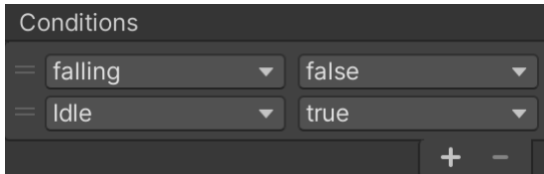


Cambiando la variable booleana jumping a true. Lo mismo para otros.

Condición de transición Jump → Fall.



Condición de transición Fall → Idle.



Claro que para funcionar todos estos, necesitamos hacer cambios en el script de Player usando las condiciones definidas anteriormente.

Primero. Se establece la variable jumping: true en la línea 49 (usando Animator: anim), pero así llega al problema de que al pulsar la tecla espacio, el carácter se queda siempre en la animación de salto y esto no es lo que queremos.

Para poder arreglar esto hay que crear una nueva función SwitchAnimation() y no olvidar poner en la función FixedUpdate().

Segundo. La función SwitchAnimation() funciona para alterar las condiciones de la transición para poder venir y llegar de forma adecuada a cada transición: si el valor de eje es negativo, significa que está cayendo y establece las condiciones de Jump → Fall; si colisiona con el terreno establece las condiciones de Fall → Idle. Lo demás funciona directamente sin script.

Observación. El script tiene añadidas tres variables más: anim, ground, Pcollider que tiene configurado con los componentes Animator, LayerMast y Collider2D para poder usar sus

funciones y propiedades. El primero es para establecer las condiciones de las variables anteriores, el segundo es para hacer un Layer de los terrenos y el último para ver las colisiones.

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class PlayerController : MonoBehaviour
6  {
7      public Rigidbody2D body;
8      public Animator anim;
9      public LayerMask ground;
10     public Collider2D Pcollider;
11     [SerializeField]
12     private float moveForced;
13     [SerializeField]
14     private float jumpForce;
15
16     // Update is called once per frame
17     void FixedUpdate()
18     {
19         Movement();
20         SwitchAnimation();
21     }
22
23     void Movement()
24     {
25         float horizontalmove = Input.GetAxis("Horizontal");
26         float facedirection = Input.GetAxisRaw("Horizontal");
27
28         if (horizontalmove != 0)
29         {
30             body.velocity = new Vector2(horizontalmove * moveForced * Time.deltaTime, body.velocity.y);
31             anim.SetFloat("running", Mathf.Abs(facedirection));
32         }
33         if (facedirection != 0)
34         {
35             transform.localScale = new Vector3(facedirection, 1, 1);
36         }
37
38         if (Input.GetButtonDown("Jump"))
39         {
40             body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
41             anim.SetBool("jumping", true);
42         }
43     }
44
45     void SwitchAnimation()
46     {
47         anim.SetBool("Idle", false);
48         if (anim.GetBool("jumping"))
49         {
50             if (body.velocity.y < 0)
51             {
52                 anim.SetBool("jumping", false);
53                 anim.SetBool("falling", true);
54             }
55         }
56         else if (Pcollider.IsTouchingLayers(ground))
57         {
58             anim.SetBool("falling", false);
59             anim.SetBool("idle", true);
60         }
61     }
62 }
63
64
65
66
67
68
69
70

```

Figure 2.10. Código C# renovado para adaptar con las animaciones.

LayerMask. Es un atributo de Unity que se usa para hacer la capa de gameObject. De esta forma, poder distinguir y usar para llamar a todos los gameObject de la misma capa en una función.

En Unity, int32 se usa para representar 32 capas. int32 significa que el sistema binario tiene un total de 32 bits (0-31). Cada GameObject tiene una propiedad Layer, y la capa predeterminada es Default. Hay 24 capas (8-31 capas) que se pueden editar en Unity, que

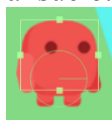
tiene ya usado oficialmente 0-7 capas, que no son editables por defecto. Por eso, LayerMask es en realidad una operación de código de bits presentando como 32 capas y no se pueden agregar.

Extra1 - Debug: Errores Móviles y Mejorar Script

Durante el desarrollo de un juego, lo normal es llegar a tener algún problema técnico no considerado anteriormente o fallado en algún proceso, denotados Bug. En esta sección extra, se estudian los errores comunes que suceden en el desarrollo de un juego y mejorar el script usando la función start.

Primer error móvil. Si se hace mover el carácter horizontalmente muchas veces, se nota que se pega en algún momento y al quitar Freeze Rotation Z, se vuela. Esto es porque el carácter y el terreno tienen el mismo collider cuadrado y colisionan debido a que están demasiado pegados (intersecan en una línea). Hay dos formas para resolver esto:

- La primera es considerar otro collider, el circle collider 2D para el carácter poniendo en la parte que pega al terreno, reduciendo el área de Box collider para que no pegue al suelo. Así llega a tener que solo intersecan en un punto.



- Otra forma es más complicada usando el script de Player, y consiste en hacer que colisionan si está fuera de la desviación de la colisión entre Player y el terreno. Para esto hay que usar la función Physics.OverlapSphere(), pero en este trabajo se usa la primera forma y no entra tanto a esta segunda forma (referencia: <https://docs.unity3d.com/ScriptReference/Physics.OverlapSphere.html>).

Además, se nota que, si el carácter colisiona en el cielo con un objeto ejecutando la fuerza constantemente, no cae debido a la fricción. Para arreglar esto, solo se necesita añadir un material sin fricción al componente collider.

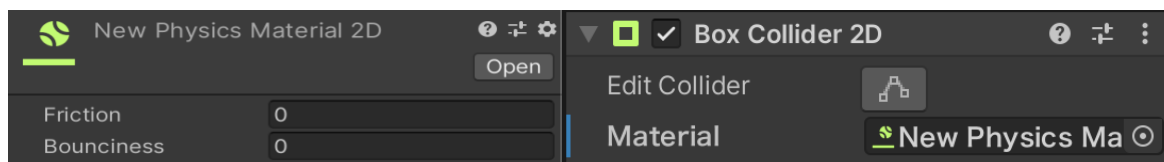


Figure 2.11.1. Nuevo material.

Segundo error móvil. Sobre el movimiento vertical, el salto. Poder hacer la acción de saltar en el cielo. Esto no es lo que se quiere para el juego. Y para arreglarlo, se cambia un poco el script.

```
50 if (Input.GetButtonDown("Jump") && Pcollider.IsTouchingLayers(ground))
51 {
52     body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
53     anim.SetBool("jumping", true);
54 }
```

Figure 2.11.1. script para no saltos infinitos.

Únicamente se ha añadido una condición más en el if-statement antes de la acción del salto.

Mejorando el script. En este paso, se introduce el uso y porqué usar **GetComponent** en Unity. Visto que en el script tiene los componentes indicados como público en vez de privado para poder mostrar y añadir el componente correspondiente. Pero en la clase es mejor considerar las variables como privado y es posible añadir automáticamente el componente usando **GetComponent** (Figure 2.11). Para esto hay que llamar a los componentes con la función **start()** o **awake()**. También puede usarse **[SerializeField]** para visualizar en Inspector de Unity.

```
7 [SerializeField]
8 private Rigidbody2D body;
9 [SerializeField]
10 private Animator anim;
11 [SerializeField]
12 private Collider2D Pcollider;
13
14 public LayerMask ground;
15 [SerializeField]
16 private float moveForced;
17 [SerializeField]
18 private float jumpForce;
19
20
21 // Start is called before the first frame update
22 void Start()
23 {
24     body = GetComponent<Rigidbody2D>();
25     anim = GetComponent<Animator>();
26     Pcollider = GetComponent<CircleCollider2D>();
27 }
```

Figure 2.11. C# script mejorado.

2.5-Control de cámara y colección de artículos

El primer objetivo de esta sección es controlar la cámara para que persiga al carácter Player, y no salir del borde de la escena. Para que funcione debe tener un script para la cámara.

```
5 public class CameraFollow : MonoBehaviour
6 {
7     private Transform player;
8     private Vector3 temPos;
9     [SerializeField]
10    private float minX, maxX;
11
12    void Start()
13    {
14        player = GameObject.FindWithTag("Player").transform;
15    }
16    void LateUpdate()
17    {
18        if (!player)
19            return;
20        temPos = transform.position;
21        temPos.x = player.position.x;
22
23        if (temPos.x < minX)
24            temPos.x = minX;
25        if (temPos.x > maxX)
26            temPos.x = maxX;
27
28        transform.position = temPos;
```

Figure 2.12. El script CameraFollow

Interpretación del script (Figure 2.12): Agregar una etiqueta para el Player para poder encontrar el gameObject Player desde otro script del gameObject Main Camera, después obteniendo la posición de la Main Camara para que iguale a la posición de Player y también hay que tener en cuenta el borde de la escena, de modo que la posición de Main Camera no supere los bordes izquierdo y derecho. Lo mismo se debe hacer para los bordes superior e inferior.

Debe tenerse en cuenta que todo esto debe estar escrito en la función LateUpdate() para que se haga después de tener llamadas las acciones de Player.

No vamos a entrar mucho a esto ya que es muy trivial de escribir en un script. Y esta sección no va a ser tan aburrida debido al Cinemachine que posee Unity.

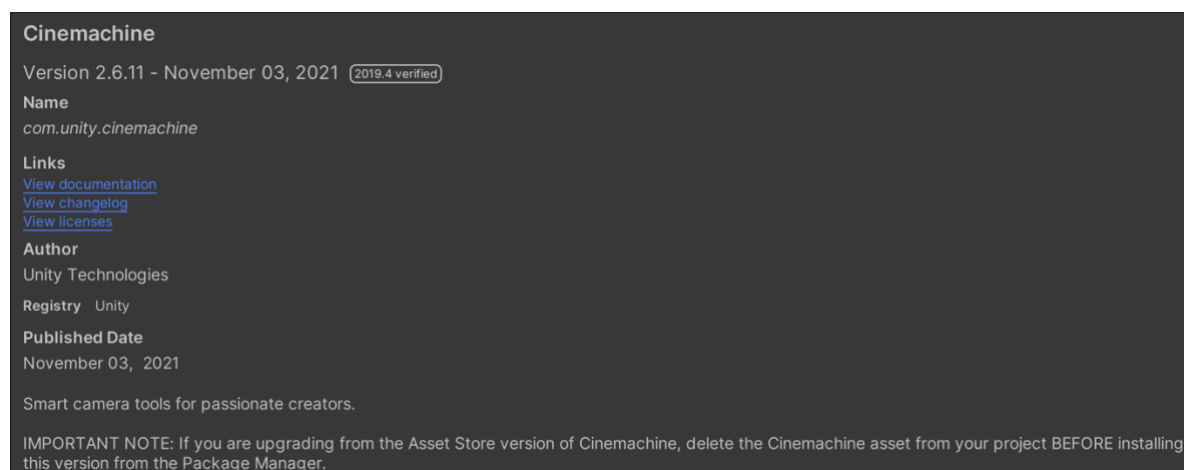
Cinemachine es una accesoria que posee Unity y funciona para editar de una forma automática la cámara, solo dando unos parámetros y aspectos que queramos. Debido a que hay una herramienta tan útil, poder adjuntar de forma sencilla para la cámara.

Paso1. De hacer que la cámara persigue al Player usando el script.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class CameraControl : MonoBehaviour
6  {
7      public Transform Tplayer;
8
9      // Update is called once per frame
10     void Update()
11     {
12         transform.position = new Vector3(Tplayer.position.x, Tplayer.position.y + 2f, -10f);
13     }
14 }
```

Figure 2.13. El script de CameraControl.

Paso2. Añadir el paquete Cinemachine que está en window → package manager → buscar cinemachine e importarlo en el menú.



Para el juego en 2D, añade una cámara 2D (CM vcam1) que se encuentra en menú → cinemachine → create 2D Camara y agrega el gameObject Player en el parámetro follow.

Luego, únicamente hay que manipular con los parámetros del Body para que funcione de la forma adecuada dependiendo del gusto de cada uno.

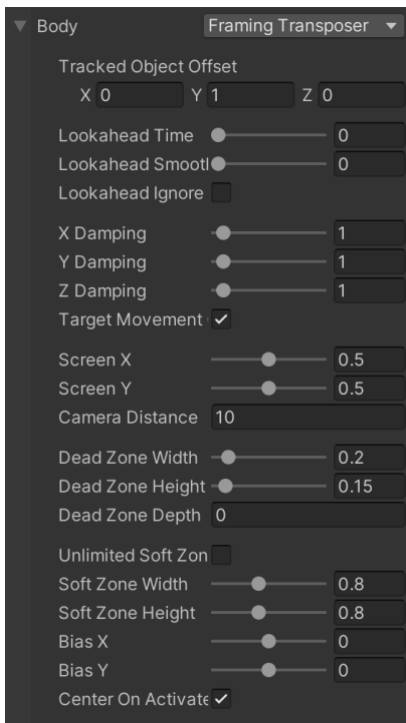


Figure 2.14. Los parámetros de Body de cinemachine 2D. (Para profundizar sobre cinemachine, visita la página oficial: <https://unity.com/es/unity/features/editor/art-and-design/cinemachine>).

En este momento, aún nos podemos salir de los bordes. Que solo tiene el aspecto de la cámara y el perseguimiento hechos. Utilizando el componente Cinemachine Confiner que pose en el Inspector de CM vcam1, se puede agregar un collider para que la cámara colisione con la funda si quiere salir de la funda. Entonces, se añade el componente Polygon Collider 2D y se adjunta con el background y se agrega el collider background en ese componente (en Bounding Shape 2D).

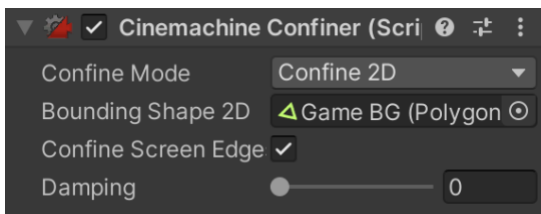


Figure 2.15. El componente Cinemachine Confiner.

ATENCIÓN: No se debe olvidar activar Is Trigger (se convierte en Trigger, mencionado en la página 10) en el collider de la funda, si no el Player colisiona con ese collider.

El segundo objetivo no está muy relacionado con el Player, pero sí que tiene que cambiar un poco de aspecto en él. Consiste en crear los artículos coleccionables. Es extremadamente sencilla una vez que están hechos todos estos.

Paso1. Renderizar los objetos colectivos en la escena y hacer la animación con la misma forma que tiene hecho para el carácter.

Paso2. Añadir el componente collider a los objetos colectivos y activar is Trigger. Luego, se crea un script para aplicar la destrucción y enumeración de los objetos colectivos si el carácter colisiona con los objetos.

```

19      public int Coin;
20
21      void Start()...
27
28      void FixedUpdate()...
33
34      void Movement()...
56
57      void SwitchAnimation()...
74
75      private void OnTriggerEnter2D(Collider2D collision)
76      {
77          if(collision.tag == "Collection")
78          {
79              Destroy(collision.gameObject);
80              Coin += 1;
81          }
82      }
83  }

```

Figure 2.16. Uso de la función OnTriggerEnter2D.

Interpretación de script: Al tener activado isTrigger, si el player colisiona con el objeto, entra a la función y activa el evento que tiene (en este caso, destruye el objeto de la escena) y hace la enumeración. Teniendo en cuenta que está escrito en el script de Player y se activa si colisiona con objetos que están etiquetado como Collection.

Extra2 – Introducción al User Interface

En la sección anterior, teníamos hecha la función (como evento) para contar los objetos coleccionados por el jugador. Ahora bien, usamos UI para mostrar la cantidad de los objetos coleccionados, así introduciendo Unity UI. Y en el siguiente capítulo vamos a usar UI para la creación de diálogos, del menú principal y del menú de la pausa.

Paso1. Esencialmente es necesario tener creado un gameObject UI en el Hierarchy, llamado canvas. Es como un lienzo blanco donde podremos añadir los textos o dibujos que queramos. En este caso, añade dos textos situados en la esquina superior izquierda (Figure 2.17), uno para poner la letra “coin” y otro para indicar el número de monedas obtenidas en el momento.

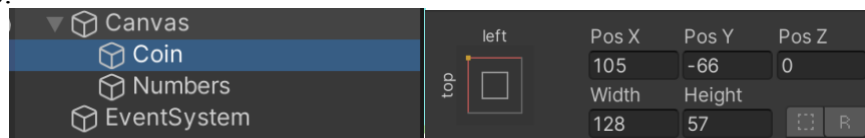


Figure 2.17. El componente Rect Transform del texto Coin.

Paso2. Hacer pequeños cambios en el script Player para poder ir actualizando el número continuamente.

```
77     public Text CoinNum;
78     private void OnTriggerEnter2D(Collider2D collision)
79     {
80
81         if(collision.tag == "Collection")
82         {
83             Destroy(collision.gameObject);
84             Coin += 1;
85             CoinNum.text = Coin.ToString();
86         }
87     }
```

Figure 2.18. La función OnTriggerEnter2D renovada.

Es de usar la variable tipo Text (CoinNum) que proviene de Unity asignado al UI-text Numbers y va igualando a la variable int Coin que tiene creado anteriormente. Teniendo en cuenta que para el uso de la variable tipo Text necesita abrir la librería UnityEngine.UI (using UnityEngine.UI;).

Capítulo 3

3E: Enemigos, Engendrador y Efectos

En este capítulo, vamos a crear el enemigo, que no va a ser de la misma forma que el jugador. E introducimos el concepto y la creación de Spawner. Después de esto, pasamos a la segunda parte de hacer eventos (Animation Events) y efectos (audio, luz, diálogo, paralaje) del juego.

3.1- Creación de enemigos

Empezando con ellos, tenemos que crear un gameObject de la misma forma que el player. Necesitamos crear un new sprite para poder renderizar en la escena (sin olvidarnos de cambiar al Layer Player), añadiendo ahora los componentes rigidbody, box collider, animation. Pero no vamos a hacer la animación de los enemigos, ya que no tenemos el recurso, además es de la misma forma que vimos antes.

En esta sección lo importante es centrarse en hacer los atributos de los enemigos con respecto al player, que son los siguientes:

1. Destrucción del enemigo

En un juego general, es imprescindible tener una forma de destruir a un enemigo. En nuestro caso, tomamos la forma más simple de super Mario. Es decir, se eliminan si el player los pisa. Para esto, es necesario sacar la función OnCollisionEnter2D.

```
100     private void OnCollisionEnter2D(Collision2D collision)
101     {
102         if (collision.gameObject.tag == "Enemy")
103         {
104             if (anim.GetBool("falling"))
105             {
106                 Destroy(collision.gameObject);
107                 body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
108                 anim.SetBool("jumping", true);
109             }
110         }
111     }
```

Figure 3.1. Función de destrucción de enemigos.

Este código está dentro de script Player. Entra a esta función si el Player colisiona con algo. En cuanto es un enemigo y el player está cayendo, entonces destruye el enemigo y el player salta una vez como efecto.

2. Efecto en el player

El enemigo también debe tener un método de hacer daño al player, efecto de hurt. Así que tomamos que el enemigo repele al player si colisionan excepto de pisar a él.

```
101 private void OnCollisionEnter2D(Collision2D collision)
102 {
103     if (collision.gameObject.tag == "Enemy")
104     {
105         if (anim.GetBool("falling"))
106         {
107             Destroy(collision.gameObject);
108             body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
109             anim.SetBool("jumping", true);
110         }
111         else if (transform.position.x < collision.gameObject.transform.position.x)
112         {
113             body.velocity = new Vector2(-5, body.velocity.y);
114             isHurt = true;
115         }
116         else if (transform.position.x > collision.gameObject.transform.position.x)
117         {
118             body.velocity = new Vector2(5, body.velocity.y);
119             isHurt = true;
120         }
121     }
122 }
```

Figure 3.2. Función renovada para el 2.

Es fácil de entender el código, pero aquí hay un pequeño problema. De esta forma no va a funcionar bien ya que en la función FixedUpdate() va actualizando frecuentemente la función Movement(). Entonces, no va a tomar nunca la velocidad que lleva en la función OnCollisionEnter2D().

Añadimos una variable booleana isHurt, para arreglar esto.

```
32 private bool isHurt; // default false
33 void FixedUpdate()
34 {
35     if (!isHurt)
36     {
37         Movement();
38     }
39     SwitchAnimation();
40 }
```

Figure 3.3. La variable isHurt

Una vez añadido necesita tener un camino de tomar otra vez isHurt como true. Con esta idea, hay que cambiar isHurt en la función que viene detrás de movement, es decir, en la función SwitchAnimation(). Cuando llega al reposo (valor absoluto de la velocidad del player es menor que 0.1), cambia el valor true a false.

```
65 void SwitchAnimation()
66 {
67     anim.SetBool("Idle", false);
68     if (anim.GetBool("jumping"))
69     {
70         if (body.velocity.y < 0)
71         {
72             anim.SetBool("jumping", false);
73             anim.SetBool("falling", true);
74         }
75     } else if (isHurt)
76     {
77         if (Mathf.Abs(body.velocity.x) < 0.1f)
78         {
79             isHurt = false;
80         }
81     }
```

También podemos hacer una animación para el efecto hurt. Desafortunadamente, tampoco tiene el fuente, pero es posible hacerlo de la forma que vimos y haciendo el anim.SetBool dentro de if (isHurt) de la función SwitchAnim().

3. Movimiento del enemigo

Sabemos que el enemigo debería tener también movimiento, pero de forma automática. No vamos a hacer que toma el movimiento como si fuese un boss del juego 3A, sino solo haciendo que mueva de izquierda a derecha y de arriba a abajo.

El primer paso es crear el script para el enemigo y dos gameObject left, right como hijos del gameObject enemy1. Poniendo estos dos objetos en los dos puntos donde el enemigo va yendo hacia izquierda y derecha, como la imagen 3.4.

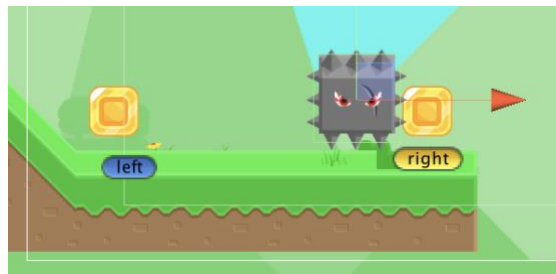


Figure 3.4. Alcance del movimiento del enemigo.

Luego, haciendo el código en el script del enemigo para que funcione este atributo.

```

5  public class Enemy1 : MonoBehaviour
6  {
7      private Rigidbody2D body;
8      public Transform leftpoint, rightpoint;
9      private bool f_left = true;
10     public float Speed;
11     private float leftx, rightx;
12
13     void Start()
14     {
15
16     }
17
18     void Update()
19     {
20
21     }
22
23     void Movement()
24     {
25         if (f_left)
26         {
27             body.velocity = new Vector2(-Speed, body.velocity.y);
28             if(transform.position.x < leftpoint.position.x)
29             {
30                 f_left = false;
31             }
32         }
33         else
34         {
35             body.velocity = new Vector2(Speed, body.velocity.y);
36             if (transform.position.x > rightpoint.position.x)
37             {
38                 f_left = true;
39             }
40         }
41     }
42 }

```

Figure 3.5. El código para el movimiento del enemigo.

Interpretación: Es muy simple. Si al tocar la posición x del objeto left, entonces cambia de dirección hacia derecha y hace una fuerza para ese lado. Lo mismo si al tocar la posición x del objeto right.

ATENCIÓN: No olvidemos generalizar el componente Rigidbody2D y dejar la función Movement() dentro de la función Update().

Con esto no es suficiente para que funcione bien el movimiento ya que los objetos left y right son objetos hijos del enemy1 y se van moviendo con enemy1. Para resolver este problema, se puede usar la función DetachChildren() para hacer libre a sus objetos hijos con respecto al transform de enemy1. Así queda en el Hierarchy con muchos gameObject left, right (no recomendado). Es mejor con la otra forma como la siguiente.

```
16 void Start()
17 {
18     body = GetComponent<Rigidbody2D>();
19     transform.DetachChildren();
20     leftx = leftpoint.position.x;
21     rightx = rightpoint.position.x;
22     Destroy(leftpoint.gameObject);
23     Destroy(rightpoint.gameObject);
24 }
25
```

Figure 3.6. Destrucción de los objetos hijos.

Tomamos dos variables para la posición x de los objetos left y right, y luego destruimos left y right.

Extra. Podemos añadir las animaciones para el movimiento de un enemigo, usando la técnica de Animation Events. Que es algo que vimos en la parte de hacer animación, pero con una red de relación más grande y con más técnicas de detectar el instante de cambiar las relaciones. No vamos a ver esto ya que no tenemos recursos para hacerlo.

3.2- Spawner (Engendrador)

.....

Un Enemy Spawner, también llamados generadores de criaturas, es un gameObject que genera criaturas (enemigos) de forma aleatoria en un período de tiempo.



Figure 3.7. Un spawner.

Representando como una bola, que es un gameObject con la propiedad del script que hace lo siguiente: Tomando una lista de las criaturas (monsterReference), inicialmente genera un enemigo en la posición de Spawner, entra a la función SpawnMonsters, que es una función enumeradora, donde va generando los enemigos en un periodo de tiempo aleatorio. Pero tiene que cumplirse que el anterior enemigo generado esté destruido.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class MonsterSpawner : MonoBehaviour
6  {
7      [SerializeField]
8      private GameObject[] monsterReference;
9
10     private GameObject spawnedMonster;
11
12     private int randomIndex;
13
14     void Start()
15     {
16         spawnedMonster = Instantiate(monsterReference[0]);
17         spawnedMonster.transform.position = transform.position;
18
19         StartCoroutine(SpawnMonsters());
20     }
21
22     IEnumerator SpawnMonsters()
23     {
24         yield return new WaitForSeconds(Random.Range(1, 5));
25
26         if (spawnedMonster == null)
27         {
28             randomIndex = Random.Range(0, monsterReference.Length);
29
30             spawnedMonster = Instantiate(monsterReference[randomIndex]);
31             spawnedMonster.transform.position = transform.position;
32
33         }
34     }
35 }
36
37 }
```

Figure 3.8. El script para Spawner.

StartCoroutine es iniciar una corrutina, y **yield return** es donde el bloque iterador regresa para llamar a la iteración. Tenemos que tomar el tipo de función IEnumerator para usar yield return.

3.3- Efectos: Audio, Dialogo y Luz.

.....

⇒ Audio

Hay tres componentes del audio que se introduce en esta subsección: Audio Listener, Audio Source, Audio Clips.

Definición (Audio Listener): El Audio Listener actúa como un dispositivo parecido a un micrófono. Este recibe input de cualquier Audio Source dado en la escena y reproduce el sonido a través de los altavoces del computador. Para la mayoría de las aplicaciones tiene mucho sentido adjuntar el listener a la cámara principal. Además, normalmente solo tiene un único Audio Listener para cada escena. [Technologies, U. *Audio Listener – Unity Manual*.].

Definición (Audio Source): Se adjunta un AudioSource a un GameObject para reproducir sonidos en un entorno 2/3D, que necesita tener un AudioListener para que funcione. Si los sonidos se reproducen en 3D o 2D está determinado por la configuración de AudioImporter. [Technologies, U. *Audio Source – Unity Manual*.].

Definición (Audio Clips): Un AudioClip almacena el archivo de audio comprimido como ogg vorbis o sin comprimir. AudioSources hace referencia a los AudioClips y los utiliza para reproducir sonidos. [Technologies, U. *Audio Clip – Unity Manual*.].

Resumiendo, podemos imaginar Audio Listener como la oreja, Audio Source es un reproductor y produce el sonido almacenado en el AudioClip. Necesita la oreja para poder oír el sonido producido en un reproductor.

Antes de empezar, es necesario ir a importar las fuentes de sonidos (de la misma forma que el de asset). Para este trabajo, tengo buscados dos paquetes de sonidos gratis en el Asset Store de Unity. Uno para el BGM del juego y otro para los sonidos de acciones.

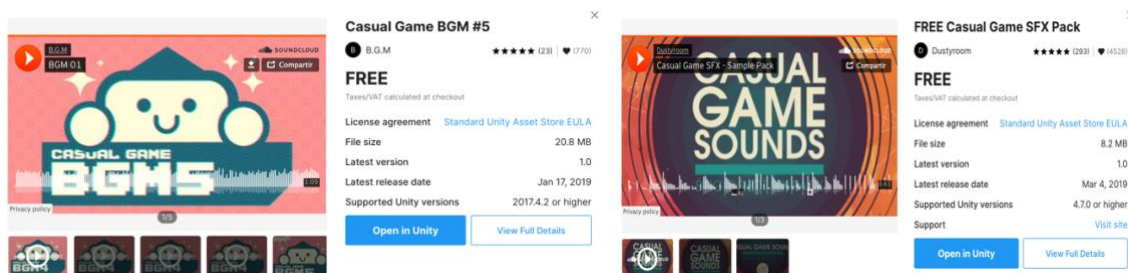
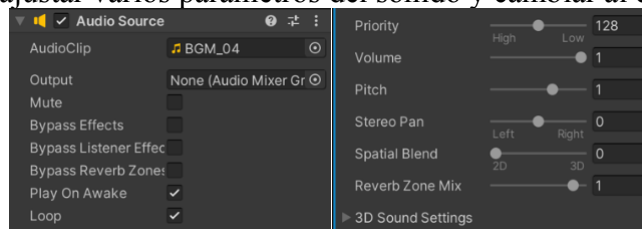


Figure 3.9. Dos paquetes de sonidos en el Unity Asset Store.

1. BGM

Por defecto, el Audio Listener está en el Inspector de la cámara principal. De esta manera, solo tenemos que añadir el componente AudioSource al gameObject al que quiere que produzca el sonido.

El BGM va siguiendo al jugador, entonces necesita añadir un componente AudioSource en el Inspector de Player. Seleccionando ahora una fuente de sonido (importado anteriormente) para el BGM, poniendo en el AudioClip, y hace que reproduzca al empezar y va repitiendo (también es posible ajustar varios parámetros del sonido y cambiar al de 3D).



2. El sonido de eliminar enemigos

Para configurar bien este sonido, es necesario usar el script de enemigos. Hay que ponerlo antes de que se elimine el enemigo, para lo que hacen falta unos pocos trucos ya que una vez eliminamos el enemigo, el componente AudioSource que lleva el enemigo desaparece.

Un truco para resolver esto es tratar de conservar el AudioSource en una variable y esto necesita tener la clase de los enemigos para crear una variable del enemigo que va a eliminar.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class Enemy : MonoBehaviour
6  {
7      protected AudioSource DeathAudio;
8      protected virtual void Start()
9      {
10         DeathAudio = GetComponent<AudioSource>();
11     }
12
13     public void Death()
14     {
15         Destroy(gameObject);
16     }
17
18     public void DeathSound()
19     {
20         DeathAudio.Play();
21     }
22 }
23
24 public class Enemy1 : Enemy
```

Figure 3.10. Clase de los enemigos

PrimerPaso. Crear un script para todos los tipos de enemigos con las propiedades compartidas (ahora mismo solo tenemos la de destrucción de los enemigos y producción del sonido cuando mueren) y hacer la herencia de esta clase para las clases concretas de cada tipo de enemigos. También se puede hacer sin añadir esta clase, pero se hace más costoso.

```
107 private void OnCollisionEnter2D(Collision2D collision)
108 {
109     if (collision.gameObject.tag == "Enemy")
110     {
111         Enemy enemy = collision.gameObject.GetComponent<Enemy>();
112         DeathAudio = enemy.GetComponent<AudioSource>();
113         if (anim.GetBool("falling"))
114         {
115             DeathAudio.Play();
116             enemy.Death();
117             body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
118             anim.SetBool("jumping", true);
119         }
120     }
121 }
```

Figure 3.11. Reproducción del sonido de la eliminación

SegundoPaso. Usando la clase de los enemigos, podemos guardar en una variable el enemigo colisionado y obtener su componente de AudioSource en la variable DeathAudio que es del tipo AudioSource. Finalmente, produce el sonido antes de que elimine con el atributo .Play().

3. Otros sonidos

Por ejemplo, el sonido de saltos. Se hace de una forma muy similar pero como es un sonido que se reproduce alrededor del Player, entonces no hace falta considerar una variable de otros gameObject. Con una sola variable pública para almacenar el sonido ya es suficiente.

El proceso sigue de esta forma: añadir un componente AudioSource para el Player, seleccionar un recurso de sonido importado para el salto y ponerlo en el AudioClip, crear una variable pública de tipo AudioSource en el script del Player, almacenar el componente AudioSource en la variable creada, y hacer que reproduzca en un instante correcto, poniendo el atributo Play () en la variable antes del código de salto en el script de Player.

```
24      public AudioSource jumpAudio, hurtAudio, CoinAudio, DeathAudio;
60      if (Input.GetButtonDown("Jump") && Pcollider.IsTouchingLayers(ground))
61      {
62          body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
63          jumpAudio.Play();
64          anim.SetBool("jumping", true);
65      }
```

Figure 3.12. El script para el sonido de salto.

De una forma análoga, se pueden configurar los sonidos de obtener una moneda y de herida.

⇒ Diálogo

El diálogo es una cosa muy esencial para el juego, debido a que es una forma muy directa de expresar algo. Por ejemplo, mostrar las instrucciones del juego. Por esta razón, vamos a ver las habilidades básicas de crear un diálogo.

Con el uso de UI es fácil de crear un panel y el texto para el diálogo. Así que creamos en el canvas (creado anteriormente) un panel con texto para mostrar el diálogo en la ventana Game. Además, es necesario ajustar los parámetros del panel y texto adecuadamente en el Inspector.

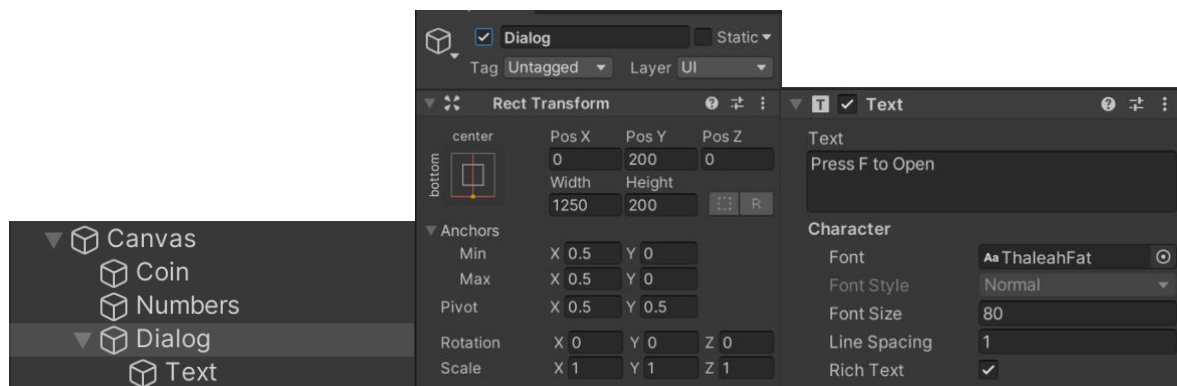


Figure 3.13. Diálogo con texto

Para dar una idea del uso, quiero hacer que el diálogo muestre en la escena del juego si el Player toca al cofre. Mostrando así la instrucción de cómo abrir el cofre.

Obviamente el cofre necesita tener un Boxcollider para hacerse como un trigger y luego escribimos la idea en un script nuevo para el cofre como la figura 3.14.



Figure 3.14. BoxCollider y script del cofre.

Haciendo el cofre como un trigger usando el componente BoxCollider, podemos usar las funciones OnTriggerEnter y OnTriggerExit para especificar si el Player toca o no al cofre. Si el Player toca al cofre, entonces se desencadena el Trigger y entra a la función OnTriggerEnter. Luego, activa el diálogo en el Inspector. En cuanto no colisiona al cofre, entrará a la función OnTriggerExit y desactiva el diálogo.

Extra: Animación del diálogo

Una cosa interesante que vamos a hacer es grabar una animación para el diálogo usando el Tab de Animation.

De la misma forma, creamos el asset Animation y Animator Controller para el diálogo. Usando el Tab de Animation, grabamos con el botón rojo que en el tiempo 0:20 aparece el panel y 0:30 aparece el texto.

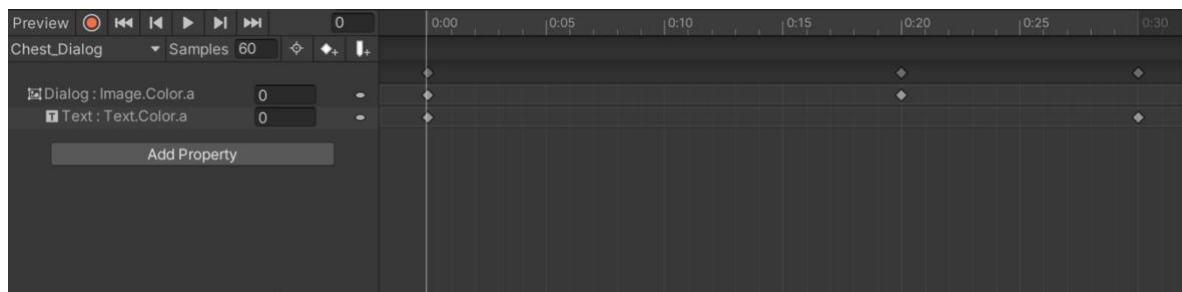
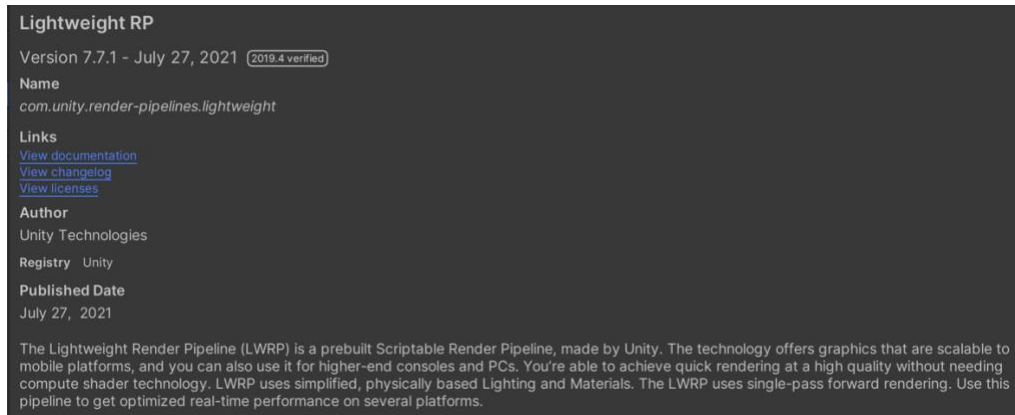


Figure 3.15. La animación para el diálogo.

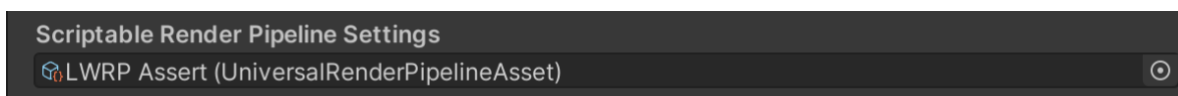
⇒ Luz 2D

La aplicación de las luces en Unity no requiere ningún script gracias a un enorme paquete dirigido a la luz. En esta subsección, vamos a familiarizarnos con algunos de los programas que proporciona.

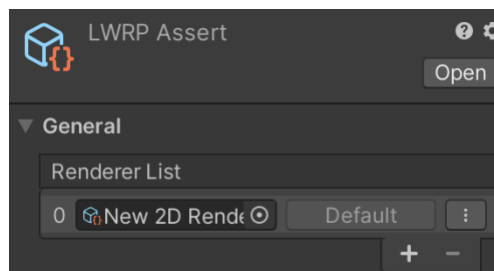
- I. Descargamos el paquete Lightweight RP (versión actual 7.7.1), que está en el window → packages → buscar Lightweight RP → click en Up to date.



- II. Hacemos que nuestro proyecto pueda usar la canalización de renderizado ligero (lightweight render pipeline). Necesitamos crear un pipeline asset (LWRP Asset) en la ventana de Project y luego ajustar en el Scriptable Render Pipeline Settings donde está en la gráfica de Project Settings.



- III. Ahora hay que ajustar este lightweight para trabajar en una plataforma 2d. Así que es necesario crear un 2D Renderer para el assert de Lightweight RP, ajustando en su Renderer List. De esta forma, Unity solo va renderizando luces 2d.



- IV. En este cuarto paso ya podemos hacer efectos en la escena. Lo primero es convertir la escena con un material nocturno para poder usar los diferentes tipos de luces. Una forma simple de añadir el material en todos los gameObject es usar el paquete LWRP (Edit → Render Pipeline → Lightweight Render Pipeline → 2D Renderer → Upgrade Scene to 2D Renderer).
- V. Por fin ya podemos usar los diferentes tipos de luces en la escena. En nuestro caso solo vamos a usar el point light 2d y el global light 2d (y a lo mejor el de Directional). Point Light, una luz que se encuentra en un punto de la escena y emite luz en todas las direcciones por igual. Y Global Light ilumina la escena de una forma globalmente.

Hay que tener en cuenta que point light y directional light tienen dimensión 3. Además, tiene otros tipos de luces con sus descripciones en el sitio web oficial de Unity: <https://docs.unity3d.com/es/2019.4/Manual/Lighting.html>.

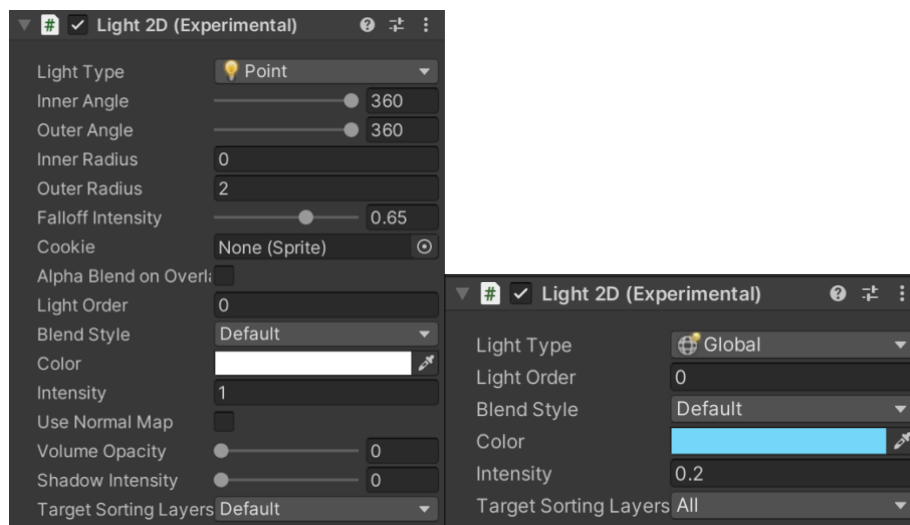
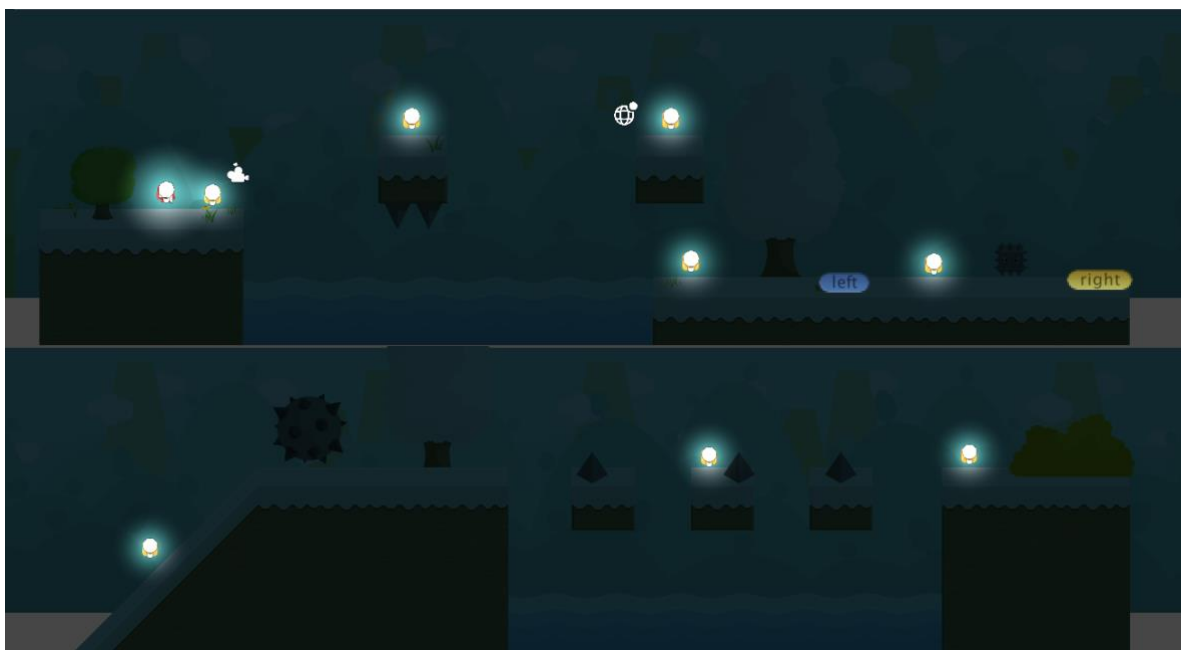


Figure 3.16. Point and Global Light.

Simplemente con ir cambiando los parámetros que tiene cada componente para ellos, es posible cambiar los efectos que proporcionan. Para más informaciones, es mejor acceder a la página web oficial: <https://docs.unity3d.com/es/2019.4/Manual/Lighting.html>.

Por la razón de la diversidad de esta subsección y para el uso de las siguientes cosas que voy a hacer, tengo creada una nueva escena (creando un asset Scena y diseñando) de una misma forma para el uso de la aplicación de las luces y otros. Tiene el aspecto de la siguiente forma:



Capítulo 4

SceneManager y Menus

Llegamos ya al último capítulo para este juego, tratando de ver la administración de las escenas, y la creación y el diseño de los menús (MainMenu y PauseMenu).

4.1- SceneManager

La administración de la escena se define como la manipulación de la escena para conseguir una alteración de la escena. En el paquete de script Unity Engine, suministra una clase SceneManager que nos facilita el control de la escena con el script.

De una forma práctica, haciendo dos propiedades para la escena, para que familiarizásemos con el SceneManager. La primera propiedad se consigue al recargar la escena una vez nuestro jugador muere de la caída. Y la segunda es el de cambio de la escena si el jugador aprueba esta escena.

1) La primera propiedad

No voy a limitar las veces que muere el jugador, pero sí que hay que conseguir llegar al final sin caer (morir). Con esta idea, necesita recargarse la escena una vez que muere el jugador, así para que empiece de nuevo, en el origen.

Transferimos la función OnTriggerEnter2D que tiene en el script de PlayerController.cs. Entonces, necesitamos un gameObject para comportarse como un trigger (cuando el jugador colisiona con este gameObject, se recarga la escena actual que tiene), poniéndolo debajo del terreno.

```
5      using UnityEngine.SceneManagement;
95     private void OnTriggerEnter2D(Collider2D collision)
96     {
97
98         if(collision.tag == "Collection")
99         {
100             CoinAudio.Play();
101             Destroy(collision.gameObject);
102             Coin += 1;
103             CoinNum.text = Coin.ToString();
104         }
105         if(collision.tag == "DeadLine")
106         {
107             GetComponent().enabled = false;
108             Invoke("Relive", 3f);
109         }
110     }
```

```

140     void Relive()
141     {
142         SceneManager.LoadScene(SceneManager.GetActiveScene().name);
143     }
144

```

Figure 4.1.1. Script para recargar la escena.

La interpretación del código de la figura 4.1.1: Tiene el efecto con lo que he dicho anteriormente, y unos efectos como el retardo de la recarga de la escena y la deshabilitación de los sonidos.

2) La segunda propiedad

Recordamos que tenemos hecho un diálogo para el cofre. Luego, tengo hecho uno para la salida de esta escena. Quiero que cuando el player esté en la salida y el jugador presione la tecla E, entonces salta de esta escena a la siguiente (es decir, entra al siguiente nivel).

Para conseguir esto, tenemos que hacer un script para el diálogo de la salida usando el paquete de SceneManager.

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.SceneManagement;
5
6  public class EnterNextScene : MonoBehaviour
7  {
8      private void Update()
9      {
10         if (Input.GetKeyDown(KeyCode.E))
11         {
12             SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex+1);
13         }
14     }
15

```

Figure 4.1.2. Script para saltar a la siguiente escena.

Cada escena tiene un índice, entonces para acceder a la siguiente escena solo necesita recargar esa escena usando su índice. Pero con una precondition de que el jugador presiona la tecla E.

Observamos que el diálogo está desactivado (activa cuando el player choca con la salida), entonces, durante el juego (sin llegar a la salida), al presionar la tecla E, no pasará nada.

Extra3 - Debug: Inflexibilidad de Jump

En cuanto estoy probando el juego, veo que el salto del Player está un poco inflexible. Por esta razón, reparamos el código de Jump.

Observamos el manual de Unity donde describe la función GetButtonDown, tiene puesto que debería de llamarlo dentro de la función Update. Además, a los movimientos que utilizan el componente Rigidbody, hay que llamarlos dentro de la función FixedUpdate ya que produce una frecuencia de sistema físico. Entendiendo con esto, deberíamos saber que hay que separar el código del movimiento jump y ponerlo dentro de la función Update().

```

45     private void Update()
46     {
47         Jump();
48     }
145 void Jump()
146 {
147     if (Input.GetButtonDown("Jump") && Pcollider.IsTouchingLayers(ground))
148     {
149         body.velocity = new Vector2(body.velocity.x, jumpForce * Time.deltaTime);
150         jumpAudio.Play();
151         anim.SetBool("jumping", true);
152     }
153 }

```

Figure 4.E.1. Jump fix code

No olvidemos poner otros movimientos en la función FixedUpdate y cambiar deltaTime a fixedDeltaTime (que es el tiempo que utiliza para la función FixedUpdate).

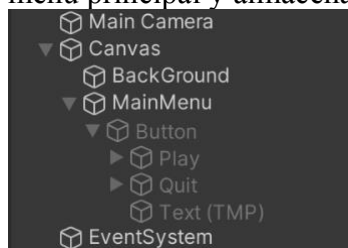
4.2 - MainMenu y PauseMenu

En esta subsección acabamos el desarrollo del juego con la construcción del menú principal y del menú de la pausa. Hasta este momento, tenemos ya introducidas y desarrolladas levemente las características de Unity. Complementamos el juego con el desarrollo de estos dos menús.

a. MainMenu

Podemos entender que el menú principal es como una escena estática, construida por un fondo, los botones y unos textos, que tienen la propiedad en cada botón. Por ejemplo, el botón de play, al pulsar, entra al juego, y el de exit sale del juego.

- i. **Primer paso:** Crear una escena nueva y añadir todos los gameObject mencionados anteriormente, la imagen para el fondo (renderizarla en un panel de UI), otro panel para añadir efectos en el menú principal y almacenar los botones y los textos.



- ii. **Segundo paso:** Diseñar nuestro menú de forma variando cada parámetro que posee Unity para el panel y el texto.

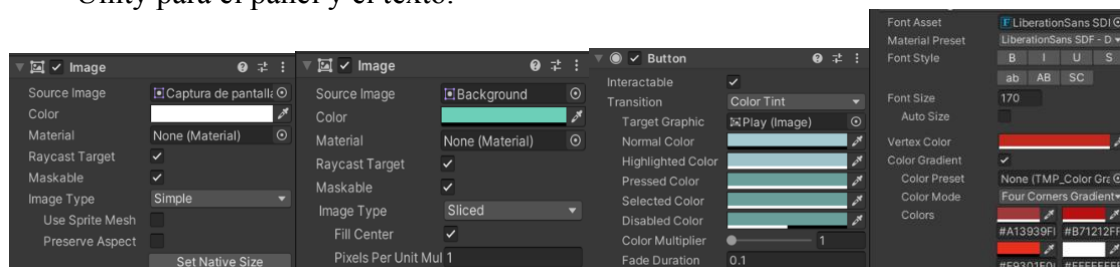


Figure 4.2.1. Los ajustes de dos paneles, un botón play y el texto del tema (por orden).

Debido a que necesitaríamos mucho espacio para explicar cada uno de estos parámetros, no vamos a entrar con ellos. Pero son fáciles de manejarlos. Presento directamente el menú que tengo diseñado.

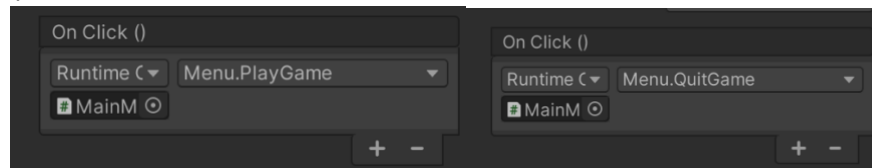


- iii. **Tercer paso:** Con estos dos botones, de play y quit, añadimos los atributos que van a tener. Trivialmente, necesitamos hacer un script para el gameObject MainMenu.

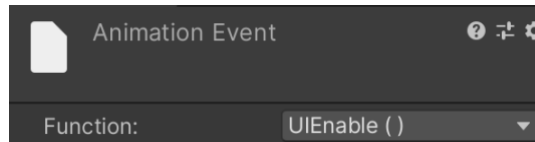
```
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5
6 public class Menu : MonoBehaviour
7 {
8     public void PlayGame()
9     {
10         SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
11     }
12     public void QuitGame()
13     {
14         Application.Quit();
15     }
16     public void UIEnable()
17     {
18         GameObject.Find("Canvas/MainMenu/Button").SetActive(true);
19     }
20 }
```

Figure 4.2.2. El script Menu para el menú principal del juego.

Necesitamos ajustar las funciones para cada botón que corresponde a la función On Click() de Inspector.



- iv. **Paso extra:** Añadiendo la animación cuando entra a la escena del menú principal. De la misma forma que para el diálogo, construyendo un delay para la aparición de los botones. Necesita usar el Animation Event, para que produzca el evento de la función UIEnable después de la animación del menú principal.



b. PauseMenu

El menú de la pausa, que vamos a crear, tiene el aspecto de la forma como la figura 4.2.3. Dentro de un panel del menú de la pausa, tenemos una barra de sonido y un botón para salir del panel y seguir con el juego. Además, en la esquina, debe tener un botón para abrir el menú y así el juego se queda detenido.

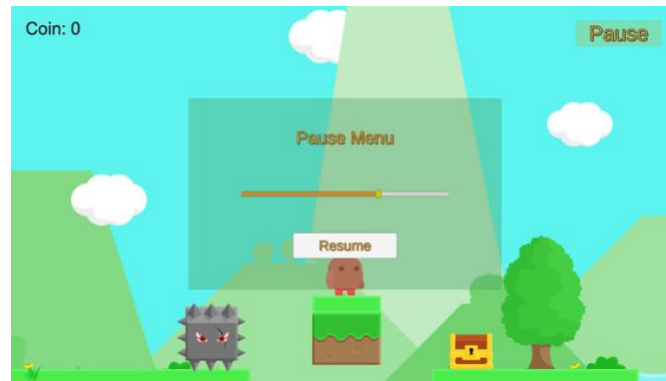


Figure 4.2.3. PauseMenu

i. Primer paso

Saltamos el paso de la creación de la apariencia del menú de la pausa, que no es nada nuevo, salvo la barra. La barra es un tipo UI que posee Unity por defecto y tiene el uso de controlar un valor, en este caso el de volumen.

Por lo que, en este paso, construimos la propiedad para el menú de la pausa, que son los que tenemos mencionados anteriormente.

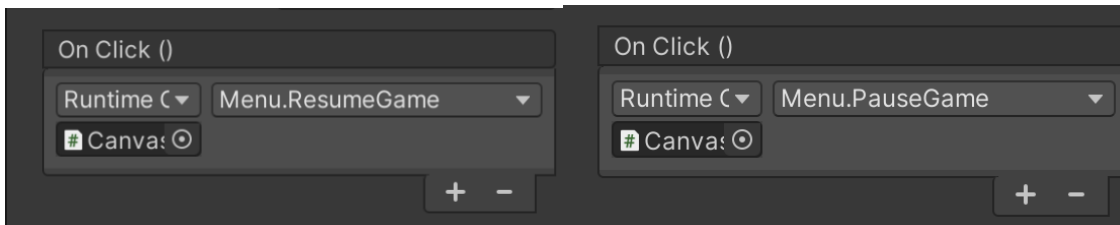
```

23     public void PauseGame()
24     {
25         pauseMenu.SetActive(true);
26         Time.timeScale = 0f;
27     }
28     public void ResumeGame()
29     {
30         pauseMenu.SetActive(false);
31         Time.timeScale = 1f;
32     }

```

Figure 4.2.4. Funciones para pausar y seguir con el juego.

Declarando las funciones a su correspondientes botones.



ii. Segundo paso

Introducimos un nuevo objeto AudioManager para poder controlar el volumen a través de la barra. Es necesario también tener una función con el valor del volumen para sincronizar el valor del volumen del AudioManager con el de la barra (hay que ajustar el máximo valor y el mínimo valor de la barra para que cuadre con el Master del AudioManager, entre -80 y 20).

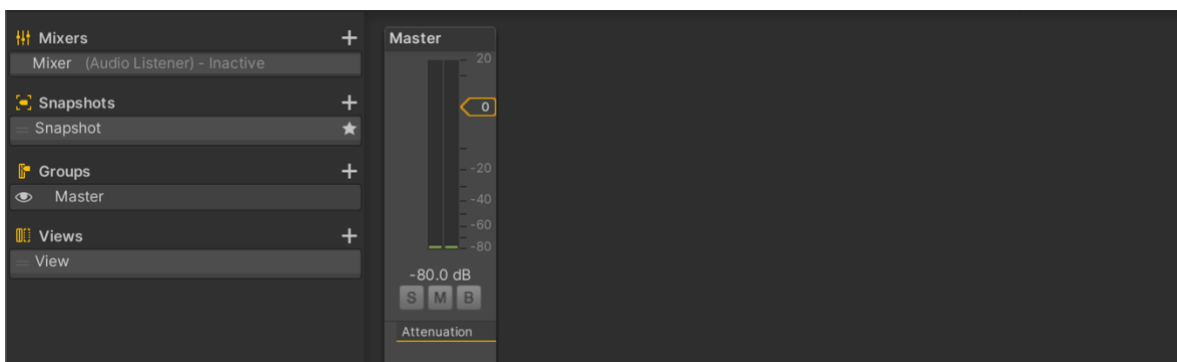


Figure 4.2.5. AudioManager

Configuramos (poniendo el AudioManager en el Output del componente AudioSource) para que el sonido salga a través del AudioManager. Y ahora sincronizamos ellos.

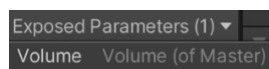
```

10      public AudioManager audioM;
33      public void SetVolume(float val)
34      {
35          audioM.SetFloat("Volume", val);
36      }

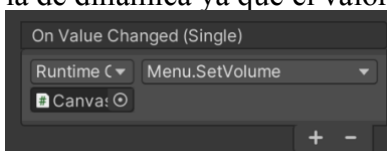
```

Figure 4.2.6. Sincronización de la barra con el componente Máster del AudioManager.

Para que funcione bien, hay que cambiar la variable volumen del Máster en una variable que puede exponer en el script, luego nombrando de una forma cómoda (en mi caso, es Volume).



Sin olvidar de añadir la función SetVolumen() dentro del OnValue() de la barra (aparecen dos funciones, una de dinámica y otra estática por parámetros, está claro que hay que usar la de dinámica ya que el valor es de tipo float cambiante).



Capítulo 5

Online Multiplayer: Photon PUN2

Este capítulo es una extensión del trabajo para incluir diversidades en el trabajo. El objetivo es usar Photon para la creación rápida de un modelo del juego tipo multi-jugador, basándonos en el juego creado anteriormente.

5.1- Introducción a Photon

Photon Engine es un motor de red eficiente, estable y escalable desarrollado por Exit Games, una empresa alemana con 15 años de experiencia en el desarrollo de back-end de servidor. Es uno de los motores de red profesionales con más usuarios y más tipos de juegos del mundo. Con Photon, los juegos de red en línea (pvp) multijugador en tiempo real se pueden realizar rápida y fácilmente.

Connect, Match and Play: PUN es una base sólida para juegos multijugador en Unity 3D para todas las arquitecturas espaciales.

5.2- Creación del modelo multijugador

• Paso 1: Importar PUN2

Necesariamente hay que crear una cuenta en la web oficial de Photon y crear una nueva app usando el Photon Cloud, Figure 5.1. Podemos observar que como máximo puede conectar 20 personas a la vez.

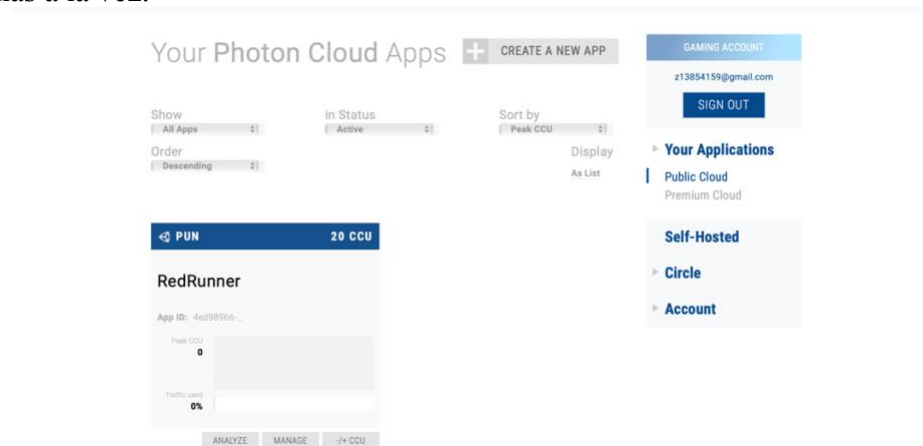
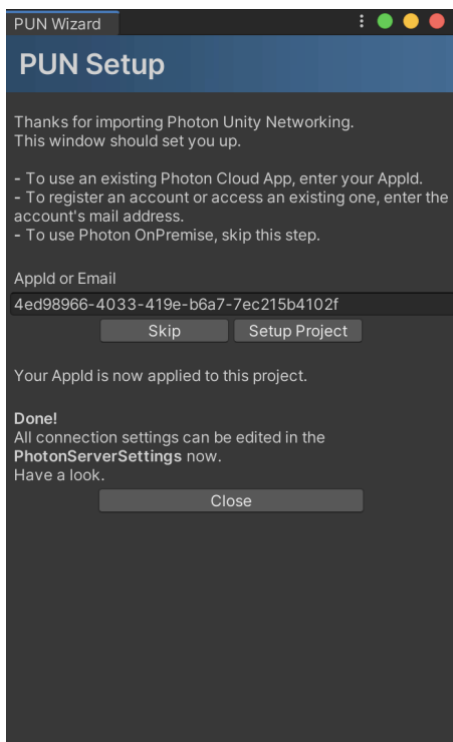


Figure 5.1. Un servidor (de Photon) para el juego RedRunner.

Luego, buscamos el paquete de PUN2 en el asset store para poder importarlo al juego.



Después de importar este paquete, sale una ventana como la figura 5.2, donde tiene que introducir la app ID del servidor RedRunner.



Configurando algunos parámetros (dependiendo del gusto), tiene ya todo preparado para poder conectar al servidor.

- **Paso 2: Conectar el juego con el servidor de Photon.**

En este paso, vamos a usar el script y un asset PhotonServerSettings que proporciona el Id para conectar al servidor.

Es necesario crear un gameObject vacío que adjunta al script (que usa para conectar al servidor) para que el script quede activado. Luego, con el código siguiente (de la figura 5.3), conecta perfectamente al servidor de Photon.

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using Photon.Pun;
5
6  public class Network : MonoBehaviourPunCallbacks // Nos da informes del servidor online.
7  {
8
9      void Start()
10     {
11         PhotonNetwork.ConnectUsingSettings(); // Significa que usa el PhotonServerSetting para conectar con el servidor.
12     }
13
14     // El siguiente código es el formato para conectar al servidor online.
15     public override void OnConnectedToMaster() // Indica si has conectado a la sala del servidor o no.
16     {
17         base.OnConnectedToMaster();
18         Debug.Log("Hola, Hanjie");
19     }
20
21
22 }

```

Figure 5.3. El script para conectar al servidor online.

- **Paso 3: Generar un Player para cada cliente.**

Una vez que ya tenemos conectado al servidor, es posible generar un Player para cada jugador(cliente) en la misma escena con su propio control de Player. Para realizarlo usamos el siguiente código:

```

15 // El siguiente código es el formato para conectar al servidor online.
16 public override void OnConnectedToMaster() // Indica si has conectado a la sala del servidor o no.
17 {
18     base.OnConnectedToMaster();
19
20     PhotonNetwork.JoinOrCreateRoom("Room", new Photon.Realtime.RoomOptions() { MaxPlayers = 5 }, default);
21 }
22
23 public override void OnJoinedRoom()
24 {
25     PhotonNetwork.Instantiate("Player", new Vector3(0,0,0), Quaternion.identity,0);
26 }

```

Figure 5.4. El código para generar un Player para cada jugador conectado al servidor.

En este código, podemos ver que primero se entra a un Room en donde se utiliza para que conecte a un mismo escenario, usando el código .JoinOrCreateRoom (nos indica que si existe el Room apropiado para el jugador, si no, se crea una nueva Room para que pueda entrar) con máximo de 5 jugadores.

Antes de generar el Player, es necesario añadir un componente Photon View para el gameObject Player. Para que, en un mismo servidor, los jugadores puedan observar a cada Player de los jugadores conectados entre sí.

Además, no debemos olvidar instanciar un Player para cada jugador que conecte al servidor y entre al Room.

Tenemos un problema de que no tiene indicado para cuál jugador se controla a un Player. Entonces, es importante indicarlo en el script de PlayerController, para cada movimiento:

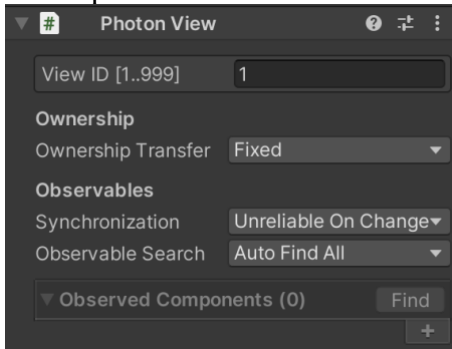
```

50     if (!photonView.IsMine && PhotonNetwork.IsConnected)
51         return;
52     Jump();

```

Cuando vea que el Player no corresponde al mío y está conectado, no realiza la función Jump. Lo mismo para todos los movimientos que controla el Player.

ATENCIÓN: Para que nuestro carácter se visualice dentro del servidor, es necesario añadir el componente Photon View.



Observamos que si el juego tiene una cámara que sigue a los jugadores, necesitamos identificar cada jugador del servidor y adjuntar la cámara adecuadamente. Y esto es fácil de resolver con PhotonView.IsMine identificando cada jugador, luego sincronizar su posición con la posición de la cámara.

```
28     void Start()
29     {
30         body = GetComponent<Rigidbody2D>();
31         anim = GetComponent<Animator>();
32         Pcollider = GetComponent<CircleCollider2D>();
33         if (photonView.IsMine)
34         {
35             gameObject.name = "Player";
36         }
37     }
```

Figure 5.5. Identificando el jugador (nombrando como player a su carácter).
Está dentro del script de PlayerController.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class CameraControl : MonoBehaviour
6  {
7      private Transform Tplayer;
8
9      private void Start()
10     {
11         GameObject player = GameObject.Find("Player");
12         if (player != null)
13         {
14             Tplayer = player.transform;
15         }
16     }
17     // Update is called once per frame
18     void Update()
19     {
20         transform.position = new Vector3(Tplayer.position.x, Tplayer.position.y + 2f, -10f);
21     }
22 }
```

Figure 5.6. Sincronizando la posición de la cámara con el de player.

También podemos hacer que se pueda rotar y varias cosas en la cámara como el tipo del juego MMORPG (a massively multiplayer online role-playing game).

5.3- Login & Name

En esta subsección mejoramos la forma de conectar al servidor. En la mayoría de los juegos, antes de entrar al servidor se tiene que introducir un id (nombre de avatar) y la sala a la cual quiere entrar. Para ello, es necesario crear dos UI para cada uno de ellos, de una forma análoga a la de creación de menú.



Después de pasar el nombre de la sala, se conecta a la sala indicada. Además, una vez introducido nuestro id, es posible nombrar individualmente a cada jugador para reconocerlos. Creamos un gameObject vacío y un archivo C# adjuntando con este gameObject para poder realizar los objetivos indicados anteriormente.

```
3  using UnityEngine;
4  using Photon.Pun;
5  using UnityEngine.UI;
6
7  public class Network_Login : MonoBehaviourPunCallbacks
8  {
9
10     public GameObject NameUI;
11     public InputField PlayerName;
12     public GameObject LoginUI;
13     public InputField RoomName;
14
15     private void Start()
16     {
17         PhotonNetwork.ConnectUsingSettings();
18     }
19
20     public override void OnConnectedToMaster()
21     {
22         NameUI.SetActive(true);
23     }
24
25     public void PlayButton()
26     {
27         NameUI.SetActive(false);
28         PhotonNetwork.NickName = PlayerName.text;
29         LoginUI.SetActive(true);
30     }
31
32     public void JoinOrCreate()
33     {
34         if(RoomName.text.Length < 2)
35             return;
36
37         LoginUI.SetActive(false);
38
39         PhotonNetwork.JoinOrCreateRoom("Room", new Photon.Realtime.RoomOptions() { MaxPlayers = 5 }, default);
40     }
41
42     public override void OnJoinedRoom()
43     {
44         base.OnJoinedRoom();
45         PhotonNetwork.LoadLevel(1);
46     }
47 }
```

Figure 5.7. C# script para Login and Name

Interpretación de figura 5.7: De una forma muy similar a la sección anterior. La diferencia está en que se han introducido dos funciones PlayButton() y JoinOrCreate(). La primera es para guardar el id dentro del servidor, desactivar el UI de nombre y activar el de la sala. La segunda es para entrar a una sala introducida por el jugador, si no hay entonces crea una nueva con ese nombre de la sala introducida, luego desactiva el UI de la sala. Después de todo esto, el jugador entra a una sala correspondiente y carga la escena inicial.

Hasta ahora tenemos hecha la primera parte, pero hace falta crear un Player en la escena inicial para cada jugador que entra a la misma sala y mostrando su id correspondiente.

De una forma análoga, se genera un carácter correspondiente a cada jugador (pero en esta nueva versión, añadido un botón play para que nuestro jugador pueda generar su carácter en cualquier momento que quiera).

```
6 public class GameManager : MonoBehaviourPunCallbacks
7 {
8     public GameObject readyButton;
9
10    public void Ready()
11    {
12        readyButton.SetActive(false);
13        PhotonNetwork.Instantiate("Player", new Vector3(1, -1, 0), Quaternion.identity, 0);
14    }
15 }
16
```

Figure 5.8. C# script para generar el carácter.

Y es posible añadir el nombre y seleccionar aspectos y otros, pero debido a falta de espacio no voy a entrar mucho con estos.

Otros. Usando PhotonNetwork.NickName lo puede lograr la representación de id y hay otros comandos más interesantes en la página web:

<https://doc.photonengine.com/en-us/pun/current/getting-started/pun-intro>.

Esta página web tiene la introducción del uso del Photon, que puede encontrar más cosas interesantes.

Capítulo 6

Conclusiones

A lo largo de los anteriores capítulos hemos desarrollado un juego (y lo hemos convertido en un juego de multijugador usando PUN3) usando Unity.

En este capítulo final presentamos las conclusiones extraídas de este trabajo sobre el desarrollo de un videojuego usando Unity. Para una mayor claridad, dividiremos en dos apartados distintos. El primero es sobre las desventajas y ventajas que aprecio al usar Unity. Y el segundo se corresponde a los trucos de cómo aprender a usar Unity por nuestra propia cuenta y los procesos que se siguen para el desarrollo de un videojuego.

6.1- Las desventajas y ventajas de Unity

Durante el desarrollo del juego, en vez de las ventajas, se ve más claro **las desventajas** que tiene en esta herramienta. Que son las siguientes:

1. Medio Ambiente

Se puede decir que el entorno está casi perfecto. La sensación de todo en uno hace que todo funcione a la perfección. Es difícil encontrar defectos. Pero hay algunos lugares que puedo decirlos.

Lo primero es que el multiproyecto no es compatible de forma predeterminada. Cada vez que se abre una escena, la escena actual se cerrará y se abrirá la siguiente escena. Y haga doble clic en el icono de inicio, se abrirá automáticamente el proyecto más reciente. Si desea resolver esto, es fácil agregar --project después del acceso directo. Después de agregar, habrá un aviso cada vez que compile, esto es irrelevante. Intente hacer doble clic en el acceso directo nuevamente y ya funciona en seleccionar los proyectos.

El segundo es que las varias versiones no pueden coexistir fácilmente (pero hay soluciones disponibles en internet).

El editor no está separado del entorno de ejecución. Para depurar, debe iniciar el editor U3D. Y muchas veces, para los programadores, no es necesario. Solo quieren cambiar el código, compilarlo y ejecutarlo. Tal vez, U3D pueda proporcionar un entorno de inicio más ligero a través de la línea de comandos.

Si hay un bucle infinito en el código, solo se puede forzar a que finalice y luego comience de nuevo. Si olvida guardarlo en el medio, el trabajo de edición de escena que acaba de hacer se borrará.

2. Gastos

Según el entorno operativo de Mono, el precio inicial (proporciona a los requisitos de rendimiento de la computadora) del motor es más alto que otros. El tiempo de inicialización es demasiado largo. Esto es inaudito para máquinas de gama media a alta. Pero las máquinas de gama baja, en comparación, parecen vergonzosas. Y esta situación, el impacto en 3D no es malo. Pero para un pequeño juego 2D, se malinterpretará que el juego no es lo suficientemente bueno.

Aunque U3D proporciona soporte 2D nativo, su esencia es bloquear la cámara en tiempo de ejecución, integrar la física BOX2D y el generador de colisiones 2D y otras cosas. La sobrecarga del precio inicial durante el tiempo de ejecución es equivalente a la de 3D.

3. Modelo de desarrollo basado en código

En los ejemplos oficiales o algunos tutoriales introductorios, se basan de una forma usando el código para un proyecto de Unity. Por supuesto, el método basado en código es realmente muy conveniente y, al mismo tiempo, es más poderoso.

Para un proyecto pequeño, el código es sin duda la forma más conveniente de operar.

Sin embargo, para un proyecto a gran escala, el resultado del código es que los errores son frecuentes y difíciles de resolver uno por uno. Al mismo tiempo, los requisitos de planificación son altos y la planificación necesita saber qué tipo de secuencias de comandos deben vincularse a diferentes objetos. Para el desarrollo basado en código, la persona que codifica es la mejor para planificar. Para diferentes objetos, use scripts para personalizar funciones específicas.

Por lo tanto, si usa U3D para proyectos a gran escala, aún debe recurrir al modelo clásico basado en datos. El modelo basado en datos simplemente no tiene los problemas que enfrenta el modelo basado en código. Al mismo tiempo, los juegos basados en datos pueden hacer mejor los juegos en línea.

Aquí, menciono brevemente los lugares que deben prestar atención al enganche de secuencias de comandos. La idea del componente de U3D es que, si el script no está adjuntado al objeto, perderá las características de este script, ahora tenemos que decidir cómo adjuntarlo. Mi idea es que, para un objeto, si se amplían sus capacidades, dicha función se puede adjuntar al objeto. Por el contrario, si se trata de determinar el flujo del juego, este debería ser un código ordinario y no se puede vincular al objeto.

4. Otros

Puedo decir que hay muchos otros inconvenientes en usar Unity, pero son todos solubles. Por lo tanto, no voy a incluirlos. Sin embargo, voy diciendo los trucos que son útiles para resolverlos.

Todos los objetos en la escena, use prefab. No pueden aparecer elementos que no sean prefab y no se pueden modificar propiedades de los elementos de prefab.

El motivo de no poder modificar las propiedades es por la sencillez de guardar, si todos tienen las mismas propiedades solo nos queda guardar su zoom, rotación, traslación y el nombre del prefab correspondiente. Cuando encuentre algo que necesite diferentes atributos, cree un nuevo PREFAB.

Extienda una función de editor para exportar la escena a un formato de texto como XML o JSON. Este formato de texto guarda toda la información del objeto, cada información del

objeto incluye zoom, rotación, traducción y el nombre prefabricado correspondiente. Por supuesto, esto no está muerto, si tiene otras necesidades, puede agregarlo usted mismo. Por ejemplo, el estado activo inicial de un objeto.

Las **ventajas** son obviamente las siguientes, también he enunciado algunos en el capítulo 1.

1. Un entorno de desarrollo integrado (IDE) personalizable

U3D tiene una idea de diseño de todo en uno, que integra todos los editores y se puede expandir libremente. Por ejemplo, si de repente desea editar el material de un modelo en la escena, seleccione el modelo, haga clic con el botón derecho y aparecerá el editor de materiales.

El pensamiento basado en componentes hace que esta relación sea aún más estrecha. Ni siquiera sientes que estás usando un editor de materiales, sientes que estás manipulando el modelo en sí, sus materiales, sus colisionadores, su estructura de objetos, etc. Y un buen motor primero debe tener un editor completamente funcional.

2. Scripts de desarrollo basados en Mono

C/C++ es sin duda el favorito del mundo de los gráficos, y nadie ha pensado en usar otro lenguaje para reemplazarlo. Sin embargo, los primeros motores escritos en C/C++ usaban C/C++ como norma para el desarrollo de la lógica de nivel superior.

Otros usan un modo de script puro, como Python y LUA. La ventaja de las secuencias de comandos son los costos de codificación más bajos, mientras que el conocido Unreal Engine y el poco conocido Torque han desarrollado un conjunto de lenguajes de desarrollo por sí mismos. Creo que su propósito es permitir que todos puedan programar de una manera más segura, y C++ puede generar problemas de memoria y eficiencia si no se tiene cuidado.

Su coste de uso y coste de personal es realmente superior al de otros lenguajes, y la irrupción de Mono C#JS y BOO ha vuelto a hacer brillar los ojos de todos. El puente de Mono permite combinar un motor de gráficos C++ eficiente con un lenguaje seguro para la memoria con GC. No solo reduce los riesgos de seguridad, sino que también facilita que todos escriban código multiplataforma.

3. Sistema de objetos basado en componentes

Simplifica la modificación y la organización de los atributos presentados en un objeto. Además, al descartarse no produce errores debido a la independencia de cada componente y su complejidad.

4. What you see Is what you get

Refiere a que un sistema en el que el software de edición edita contenido que tiene un aspecto similar al que tendría cuando se imprimiera o se convirtiera en un producto terminado, como un documento impreso, una página web o una presentación de diapositivas. WYSIWYG significa que la interfaz de usuario permite a los usuarios crear un documento mientras ven algo muy similar al resultado final. A menudo, lo que ve es lo que obtiene significa poder manipular directamente el diseño de un documento sin tener que escribir o recordar los nombres de varios comandos de diseño.

Además de ejecutarse inmediatamente después de la edición, U3D también puede editar y ver el efecto en tiempo real durante el proceso de ejecución. Por supuesto, los datos del objeto editado durante el proceso en ejecución no serán válidos después de la detención.

5. Lanzamiento multiplataforma

La plataforma compatible con el desarrollo de juegos unity3d es, sin duda, la plataforma más popular en este momento. Satisfacer las necesidades de la mayoría de los proyectos. Los primeros motores se basaban principalmente en PC y CONSOLA. El soporte WINDOWS, XBOX, PS2 está completamente bueno en Unity. La conveniente función de publicación multiplataforma de U3D también lo convierte en una de las razones por las que actualmente es el motor más rentable.

6. Buen ecosistema

La aparición de AssetStore no solo hace que el ecosistema U3D sea más estable, sino que también brinda muchas oportunidades. Puede crear complementos y venderlos en línea para obtener algunos beneficios, o puede comprar complementos de otras personas para uso o referencia. A veces, comprar algunos complementos puede sacarlo rápidamente de su situación actual. Uno es resolver el problema del progreso y el otro es resolver el problema del pensamiento, que antes no estaba disponible en otros motores.

6.2- Principiantes usando Unity

.....

Durante este trabajo, he concluido una forma de aprender el desarrollo de los videojuegos. Normalmente, he seguido los siguientes 4 pasos:

1. Aprender siguiendo los tutoriales

En principio, recomiendo seguir los tutoriales que incluye la web oficial de Unity. Luego, hay muchísimos tutoriales colgados en YouTube y otros streamings.

La ventaja de ver un tutorial es que puedes verlo una y otra vez, detenerte a pensar y actuar. Además, también se puede discutir con los internautas que están estudiando en el área de comentarios debajo del video para ver si alguien tiene el mismo problema que usted y cómo lo resolvió. Además, incluso puede encontrar mejores soluciones que las del video.

2. Aprender un lenguaje de programación sistemáticamente

Cada herramienta para el desarrollo del videojuego tiene los lenguajes de programación que adapta, es necesario reconocerlo, saber usarlo, pero no hace falta dominarlo. Por lo tanto, recomiendo a los que no han usado el lenguaje de programación que adapta a la herramienta, ver un vistazo a los tutoriales de ese lenguaje de programación.

3. Aprender varios conceptos de juego.

En distintos juegos hay varios conceptos aplicados en él y es bastante útil usarlos en el juego que vamos a desarrollar, como rigidbody, collider, spawner, etc. Pero no te apegues al concepto, solo entiende para qué se usa.

Recomiendo usar el manual oficial de Unity para entender los conceptos, que explica de una forma muy clara y completa. Unity tiene dos manuales oficiales, uno es [Manual de usuario de Unity] y el otro es [Documentación de la API de C#] para Unity. Puede encontrar estos dos manuales en el sitio web oficial de Unity, o puede descargar el manual y vincularlo a su propio Unity. El manual no debe leerse desde el principio hasta el final, y los puntos de conocimiento a menudo están más allá de la mente. La forma correcta de usar el manual debe ser buscar las palabras clave correspondientes en el manual de Unity cuando se encuentren problemas en el desarrollo real del juego.

4. Haz tú mismo un juego

Puede ser un juego totalmente copiado o un juego nuevo propiamente hecho. Lo importante es ponerse en mano a usar esos conceptos que hemos aprendido para ver los detalles y problemas que produce al usarlo.

Empezando desde un juego simple a uno que sea un poco complicado, por ejemplo, Hollow Knight, Celeste, Dead Cells, etc. Busque las sutilezas del diseño de este juego, la capacidad de profundizar en los detalles y grabarlo.

Durante el desarrollo del juego es posible que necesite un libro tutorial físico, os recomiendo dos libros bastante útiles. El [UNITY Y C# Desarrollo De Videojuegos – Ruelas Luis] y otro [Unity 3D – Marc Lidon] para los juegos 3D únicamente.

Bibliografía

[UnityM21] Unity, D. (marzo de 2021). Unity User Manual 2021.3 (LTS). Unity Manual. Recuperado 22 de junio de 2022, <https://docs.unity3d.com/Manual/index.html>

[ScriAPI21] Unity, D. (marzo de 2021). Unity Scripting API version 2021.3. Unity Scripting Reference. Recuperado 22 de junio de 2022, website: <https://docs.unity3d.com/ScriptReference/>

[UnityYC#] Luis Ruelas. (2018) UNITY y C# DESARROLLO DE VIDEOJUEGOS. Grupo Editorial RA-MA

[UNITY] Carlo I. Lopez. (2019). UNITY. Aprende a desarrollar videojuegos. Editorial: RC LIBROS (SC LIBRO)

[C#Unity] Norton, T. (2013). *Learning c# by developing games with unity 3d beginner's c guide : learn the fundamentals of c# to create scripts for your gameobjects*. Packt Publishing. Retrieved 2022.

[UnityMG] R. Stagner, A. (2013). *Unity multiplayer games*. Packt Publishing. Retrieved 2022.

[Photon] Photon, E. (2021). Photon Documentation. Introduction of Photon. Recuperado 22 de junio de 2022, website: <https://doc.photonengine.com/en-us/pun/current/getting-started/pun-intro>

web oficial de Unity para aprendizaje: <https://unity.com/es/learn>.